

VISVESVARAYA TECHNOLOGICAL UNIVERSITY
“Jnana Sangama”, Belagavi-590014, KARNATAKA, INDIA



Project Phase I - BCS685

ACIS-X: Autonomous Credit Intelligence System

*Submitted in Partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering in Computer Science and Engineering** of Visvesvaraya Technological University, Belagavi.*

Submitted by:

NITIN BHARADWAJ M V S(1AM23CS125)

NIKHIL NAGENDRA(1AM23CS120)

JOHAN K GEORGE(1AM23CS075)

PRASANNA B ACHARYA(1AM23CS149)

Under the Guidance of:

Prof. Bhoomi Joshi

Asst Professor, Dept. of CSE



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
AMC ENGINEERING COLLEGE

18th K.M. Bannerghatta Main Road,

Bengaluru – 560083

Year 2025-26

AMC ENGINEERING COLLEGE

18th K.M, Bannerghatta Main Road, Bengaluru-560083
2025-26

Department of Computer Science and Engineering



CERTIFICATE

Certified that the project report entitled **“ACIS-X: Autonomous Credit Intelligence System”** carried out by **NITIN BHARADWAJ M V S (1AM23CS125)**, **NIKHIL NAGENDRA (1AM23CS120)**, **JOHAN K GEORGE (1AM23CS075)**, and **PRASANNA B ACHARYA (1AM23CS149)**, bonafide students of AMC Engineering College in partial fulfillment for the award of Bachelor of Engineering in Computer Science & Engineering of the Visvesvaraya Technological University, Belagavi during the year 2025-26. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the Report. The project report has been approved as it satisfies the academic requirements in respect of Project work prescribed for the said degree.

Project Guide
Prof. Bhoomi Joshi,
Assistant Professor,
Dept. of CSE,
AMCEC

HoD
Dr. V. Mareeswari,
Prof and Head,
Dept. of CSE,
AMCEC

Principal
Dr. Yuvaraju B N,
Principal,
AMCEC

External Viva

External Name

Sign with Date

- 1.
- 2.

DECLARATION

We the undersigned students of 6th semester Department of Computer Science & Engineering, AMC Engineering College, declare that our project work entitled “**ACIS-X: Autonomous Credit Intelligence System**” is a bonafide work of ours. Our project is neither a copy nor by means a modification of any other engineering project. We also declare that this project was not entitled for submission to any other university in the past and shall remain the only submission made and will not be submitted by us to any other university in the future.

Name	USN	Signature
NITIN BHARADWAJ M V S	1AM23CS125	_____
NIKHIL NAGENDRA	1AM23CS120	_____
JOHAN K GEORGE	1AM23CS075	_____
PRASANNA B ACHARYA	1AM23CS149	_____

ACKNOWLEDGEMENT

First and foremost we would like to thank GOD, the Almighty for being so merciful on us. We have a great pleasure in expressing our deep sense of gratitude to founder **Chairman Dr. K.R. Paramahansa** and Executive Vice President **Mr. Rahul Kalluri** for having provided us with a great infrastructure and well-furnished labs for successful completion of our Project.

We express our sincere thanks and gratitude to our Principal **Dr. Yuvaraju B N** for providing us all the necessary support successful completion of our project.

We express our special thanks and gratitude to our Academic advisor **Dr. Nagaraja R** for providing us all the necessary advice for successful completion of our project.

We would like to extend our special thanks to **Dr. V. Mareeswari** Professor and HOD, Department of CSE, for her support and encouragement and suggestions given to us in the course of our project work.

We would like to extend our special thanks to Project coordinators **Prof. Velvizhi Ramya R** and **Prof Snigdha Kesh**, Department of CSE, for their support and encouragement and suggestions given to us in the course of our project work.

We are grateful to our guide **Prof. Bhoomi Joshi** Asst. Prof., Department of CSE, AMC Engineering College, Bengaluru for her constant motivation & timely help, encouragement and suggestion.

Last but not the least, we wish to thank all the teaching & non-teaching staffs of department of Computer Science and Engineering, for their support, patience and endurance shown during the preparation of this project report.

NITIN BHARADWAJ M V S (1AM23CS125)

NIKHIL NAGENDRA (1AM23CS120)

JOHAN K GEORGE (1AM23CS075)

PRASANNA B ACHARYA (1AM23CS149)

ABSTRACT

The Intelligent Collections System (ACIS-X) represents a modern, data-driven approach to debt recovery operations. Designed to overcome the limitations of traditional, rigid collection strategies, ACIS-X utilizes multi-agent reinforcement learning (MARL) alongside scalable backend microservices and responsive frontends. By dynamically adjusting communication cadence and tone based on simulated debtor behavior, it identifies optimal, empathetic collection strategies that maximize recovery rates while maintaining a positive customer relationship. The proposed architecture consists of discrete Python-based heuristic agents coordinating through Apache Kafka, governed through an extensible registry, and supervised to minimize operational risk and compliance drift.

CONTENTS

Certificate	i
Declaration	ii
Acknowledgement	iii
Abstract	iv
1 INTRODUCTION	1
1.1 Overview	1
1.2 Objectives	2
1.3 Purpose, Scope, and Applicability	3
1.3.1 Purpose	3
1.3.2 Scope	3
1.3.3 Applicability	4
1.4 Organization of the Report	4
2 LITERATURE SURVEY	7
2.1 Introduction	7
2.2 Objective of Literature Survey	9
2.3 Summary of Literature Survey	10
2.3.1 Scalability and Real-Time Detection Challenges	16
2.4 Drawbacks of Existing Systems	17
2.5 Problem Statement	18
2.6 Proposed Solution	20
2.6.1 Event and State Intelligence Layer	20
2.6.2 Risk and Policy Intelligence Layer	21
2.6.3 Real-Time Actioning and Monitoring Layer	22
3 REQUIREMENT ENGINEERING	24
3.1 Software and Hardware Tools Used	24
3.1.1 Software Requirements	24
3.1.2 Hardware Requirements	25
3.2 Conceptual/Analysis Modeling	25
3.2.1 Use Case Diagram	25
3.2.2 Sequence Diagram	26

3.2.3	Activity Diagram	27
3.2.4	State Chart Diagram	28
3.3	Software Requirement Specification	29
3.3.1	Functional Requirements	29
3.3.2	Non-Functional Requirements	29
4	PROJECT PLANNING	30
4.1	Project Planning	30
4.1.1	Scheduling	31
5	SYSTEM DESIGN	33
5.1	System Architecture	33
5.2	Component Design / Module Decomposition	34
5.3	Interface Design	37
5.4	Data Structure Design	39
5.5	Algorithm Design	40
5.5.1	Event Ingestion and Persistence Algorithm	40
5.5.2	Customer Metrics Computation Algorithm	41
5.5.3	External Enrichment and Risk Aggregation Algorithm	41
5.5.4	Payment Prediction and Final Risk Scoring Algorithm	41
5.5.5	Collections Action and Self-Healing Algorithm	42
6	IMPLEMENTATION	43
6.1	Implementation Approaches	43
6.2	Code Snippets	46
6.2.1	Event Ingestion and State Management Module	46
6.2.2	Machine Learning Model Training and Inference Module	47
6.2.3	Macroeconomic Stress Testing and Scenario Simulation Module	48
6.2.4	Autonomous Self-Healing and Model Governance Module	48
7	TESTING	50
7.1	Testing Approach	50
7.2	Definition and Types of Testing	51
7.3	Test Case Analysis	53
7.4	Test Cases for the Project	54
7.5	Test Results Summary	57

7.6	Testing Limitations	59
8	RESULT AND DISCUSSION	61
8.1	Result	61
8.1.1	Pipeline Latency and Lifecycle Completion	61
8.1.2	Risk Score Monotonicity	62
8.1.3	Throughput Scaling with Replicas	63
8.2	Discussion	63
9	CONCLUSION, APPLICATIONS AND FUTURE SCOPE	66
9.1	Conclusion	66
9.2	Applications	67
9.3	Future Scope	68
	ANNEXURE	70
A.1	MAJOR PROJECT Mapping with CO and PO	70
A.1.1	Strength of CO MAddresses ethical considerations in automated collectionsapping to PO/PSOs with Justification	70
A.2	SDG – Sustainable Development Goals mapping with project	73
A.3	SWOT Analysis	73
A.3.1	Strengths	73
A.3.2	Weaknesses	74
A.3.3	Opportunities	74
A.3.4	Threats	74
	ANNEXURE B — Conference Paper Acceptance Letter	75
	REFERENCES	77

LIST OF FIGURES

Fig. No.	Figure Name	Page No.
3.1	Use case diagram for ACIS-X	25
3.2	Sequence diagram illustrating event communication flows	26
3.3	Activity diagram mapping out the decision pipeline	27
3.4	State chart diagram specifying account state transitions	28
5.1	System architecture of ACIS-X	33
5.2	Component design of ACIS-X	34
5.3	Interface design of ACIS-X	37
6.1	CustomerStateAgent Kafka Consumer Group Initialization	46
6.2	Per-Customer Lock and Bulk Invoice Query	47
6.3	2-Stage Policy Guardrail Composition in PaymentPredictionAgent	47
6.4	Latent Stress SDE Step Function	48
6.5	Population Stability Index (PSI) Core Computation	48
7.1	Schema-version and dead-letter validation assertions	52
7.2	Invoice lifecycle trace through agent stages	54
7.3	P95 pipeline latency measurement	57
7.4	Hard-failure recovery assertion for a timed-out agent	59
8.1	Pipeline P95 Latency Across 50 Injection Events	61
8.2	Invoice Lifecycle Stage Waterfall (CSA → PPA → RSA)	62
8.3	Risk Score Monotonicity Across Three Behavioural Axes	62
8.4	Throughput Scaling with Consumer Group Replicas	63

LIST OF TABLES

Table No.	Table Name	Page No.
3.1	Software Requirements	24
3.2	Hardware Requirements	25
4.1	ACIS-X Project Schedule (8-Week Plan)	32
5.1	Core Components of ACIS-X	35
5.2	Kafka Topic Interface Summary	38
5.3	Data Structures Used in ACIS-X	39
6.1	Specifications of available tools	43
7.1	Testing categories used for ACIS-X	51
7.2	Representative ACIS-X test cases	54
7.3	Recorded ACIS-X test execution summary	58
A.1	Course Outcomes (CO) Statements	70
A.2	CO-PO/PSO Mapping Matrix	71
A.3	Justification for CO-PO/PSO Mapping	72
A.4	Sustainable Development Goals (SDG) Mapping	73

CHAPTER 1

INTRODUCTION

1.1 Overview

Accounts receivable operations are central to business cash flow, yet they are often fragmented across invoicing systems, payment records, external risk signals, and manual collections workflows. In many organizations, collections decisions are delayed because financial behavior signals are distributed across multiple tools and teams, making it difficult to move from raw transaction data to timely action. This delay increases overdue exposure, reduces recoverability, and creates avoidable operational risk.

ACIS-X addresses this gap through an event-driven, multi-agent intelligent collections platform that continuously ingests customer, invoice, and payment events; enriches them with external financial and litigation context; predicts payment risk; and routes high-risk situations to policy and collections actions. Instead of treating credit monitoring, risk assessment, and collections as disconnected processes, ACIS-X unifies them in a streaming architecture where each stage contributes structured intelligence to the next stage.

At the system level, ACIS-X combines:

- A Kafka-based event backbone for decoupled, scalable inter-agent communication.
- Specialized software agents for metrics computation, enrichment, aggregation, prediction, risk scoring, and action orchestration.
- A persistent storage layer centered on SQLite with query and memory support for state access and recovery.
- Operational control-plane services for runtime orchestration, placement, registry tracking, monitoring, and self-healing.

From an analytical perspective, the platform transforms a basic receivables pipeline into a feedback-driven decision system. Invoice and payment updates trigger metric recomputation; metrics trigger enrichment and predictive logic; risk outputs trigger policy and collections actions; and all stages emit telemetry for health and recovery decisions. This produces a closed-loop workflow that is continuously updated as new events arrive.

A key design characteristic of ACIS-X is role separation among agents. Context-

building components, such as customer profile and enrichment agents, gather and organize information, while decision-authority components, such as risk scoring and collections agents, publish final actionable outcomes. This separation improves maintainability, traceability, and operational clarity.

1.2 Objectives

The project objectives of ACIS-X are to build a reliable and extensible platform for intelligent receivables operations with real-time risk awareness and actionable outcomes. The primary objectives are:

- **Establish a unified event-driven collections pipeline:** Integrate customer, invoice, payment, enrichment, prediction, and collections stages through topic-based event flow to remove siloed processing.
- **Compute operational customer metrics continuously:** Derive and maintain payment-behavior indicators such as outstanding exposure, payment delay patterns, and on-time ratios from live invoice and payment events.
- **Incorporate external risk intelligence:** Enrich customer context using external financial and litigation indicators so internal behavior is evaluated alongside broader market and legal signals.
- **Generate invoice-level predictive risk signals:** Use prediction logic to estimate payment risk on open invoices and provide structured reasons and confidence attributes for downstream evaluation.
- **Publish authoritative final risk decisions:** Refine base prediction outputs using customer context and temporal behavior to emit standardized final risk-scored events.
- **Automate policy and collections actions:** Convert final risk outcomes into prioritized collection actions with duplicate prevention and escalation logic based on severity and customer state.
- **Maintain resilient runtime operations:** Provide monitoring, registry-based discovery, placement, and self-healing loops so the platform can detect degraded behavior and initiate recovery actions.
- **Support testability and maintainability:** Ensure architecture and schema behavior

can be validated using unit-focused testing patterns without requiring a live broker for every validation cycle.

- **Enable future architectural extensions:** Keep the system modular for subsequent additions such as richer policy engines, expanded external sources, advanced forecasting, and stronger observability analytics.

1.3 Purpose, Scope, and Applicability

1.3.1 Purpose

The purpose of ACIS-X is to provide a technically grounded and operationally practical framework for proactive collections intelligence. Rather than reacting after receivables have materially aged, the system is designed to surface emerging risk earlier, link that risk to customer and invoice context, and trigger consistent action paths through policy and collections workflows.

This purpose is achieved through a layered architecture where each agent contributes a well-defined transformation. As a result, ACIS-X acts as both a data-processing system and an execution pipeline for receivables risk management.

1.3.2 Scope

The current implementation scope of ACIS-X includes end-to-end event orchestration for intelligent collections with the following technical boundaries and capabilities:

- **Core domain streams:** Customer, invoice, payment, metrics, predictions, risk, collections, and system-control event streams.
- **Agent ecosystem:** Distinct agents for scenario generation, customer state, external enrichment, aggregation, profile construction, payment prediction, risk scoring, overdue detection, collections decisions, policy signaling, database persistence, memory state, and query access.
- **Runtime and control plane:** Registry, placement, runtime manager, monitoring, and self-healing components operating through events for orchestration and resilience.
- **Persistence and query layer:** SQLite-backed storage for customers, invoices, payments, metrics, risk profiles, and collections logs, with read-oriented query methods and in-memory derived state support.

- **Operational support assets:** Deployment and testing guidance, startup/reset flows, and architecture validation tests targeting contract correctness and critical design fixes.

The present scope does not represent a complete enterprise deployment stack in all dimensions. For example, storage and infrastructure are currently optimized for the existing project context, and broader production-hardening concerns such as multi-datacenter persistence, strict governance integration, and enterprise IAM policies are outside the immediate implementation baseline. However, the design remains extensible toward those directions.

1.3.3 Applicability

ACIS-X is applicable to organizations and environments where receivables risk must be identified and acted upon quickly, and where static rule-only workflows are insufficient. Typical applicability contexts include:

- **B2B enterprises with invoice-based revenue:** Companies needing continuous visibility into overdue exposure, payment volatility, and customer-level risk concentration.
- **Finance and shared services operations:** Teams seeking automated prioritization of follow-ups, escalations, and account interventions.
- **Credit risk and collections functions:** Organizations requiring explainable event trails from source transaction through risk assessment to action decision.
- **Technology teams building autonomous workflow pipelines:** Projects requiring modular, event-driven orchestration with resilient runtime behavior and observable health signals.
- **Research and prototyping environments:** Contexts where scenario generation, controlled architecture experimentation, and repeatable unit validation are needed before scaling.

Therefore, ACIS-X is best viewed as a practical intelligent-collections foundation that can support both operational usage and iterative evolution into broader financial risk automation platforms.

1.4 Organization of the Report

This report follows the standardized project documentation structure while detailing all technical aspects and contributions of ACIS-X. The chapter organization is as follows:

i. Chapter 1: Introduction

Presents the project background, motivation, primary objectives, scope boundaries, and the organizational structure of the report for ACIS-X.

ii. Chapter 2: Literature Survey

Reviews relevant research and industrial approaches in credit scoring, streaming analytics, and multi-agent systems, highlights limitations of traditional systems, and establishes the formal problem statement and proposed solution.

iii. Chapter 3: Requirement Engineering

Outlines software and hardware requirements, conceptual and analysis modeling diagrams (Use Case, Sequence, Activity, and State Chart diagrams), and comprehensive Software Requirement Specifications covering functional and non-functional requirements.

iv. Chapter 4: Project Planning

Describes the planning and scheduling framework, sprint breakdowns, execution phases, milestone deliverables, and risk mitigation strategies used across the project lifecycle.

v. Chapter 5: System Design

Details the end-to-end system architecture, component decomposition across Kafka topics and agents, interface designs (BFF API and SSE streams), relational and streaming data structures, and core algorithm designs (signal fusion, Random Forest classification, self-healing circuit breakers, and drift monitoring).

vi. Chapter 6: Implementation

Details the practical implementation methodologies, asynchronous event runtime, database access optimizations, synthetic macroeconomic stress generator, and core system modules alongside illustrative code snippets and efficiency analysis.

vii. Chapter 7: Testing

Presents the testing strategy and definitions of testing types (unit, contract, integration, lifecycle, performance, intelligence, and resilience), comprehensive test case specifications evaluating PASS and FAIL conditions, and explicit realism boundaries.

viii. Chapter 8: Result and Discussion

Presents empirical evaluation results including pipeline latency, invoice lifecycle stage waterfalls, risk score monotonicity verification, and consumer-group throughput scaling, followed by an in-depth discussion of operational findings.

ix. Chapter 9: Conclusion, Applications and Future Scope

Summarizes the key achievements and architectural validation of ACIS-X, surveys industrial applications across financial and regulatory domains, and outlines future enhancements such as live ERP integration and distributed deployments.

x. References

Compiles all cited academic publications, books, technical standards, and documentation resources in standard IEEE citation format.

xi. Annexures

Provides supplementary project documentation, including Annexure A (SWOT Analysis and Project Mapping with Course Outcomes and Program Outcomes) and Annexure B (Conference Paper Acceptance Letter).

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction

The design of intelligent collections systems has shifted from periodic scorecard workflows to continuously updated, event-driven decision pipelines. In enterprise receivables environments, late payments are not isolated events; they are outcomes of evolving customer behavior, changing macro conditions, policy execution timing, data quality variation, and process latency. As a result, any practical platform must combine accurate risk estimation with low-latency orchestration and actionable governance.

Historically, credit-risk analytics has progressed from classical statistical classification toward data-intensive machine learning and ensemble modeling. This progression has improved predictive discrimination, but it has also introduced new deployment concerns: calibration drift, model opacity, benchmark fragility, and operational disconnect between analytics and actioning. In collections operations, these gaps are particularly costly because decision delays can reduce recoverability, and unexplained interventions can reduce operational trust.

Early research in consumer credit scoring established core classification principles and highlighted domain-specific concerns such as reject inference, class imbalance, and operational validation [2, 3]. Subsequent studies benchmarked machine-learning methods and demonstrated that model effectiveness depends strongly on dataset characteristics, evaluation design, and business context [4, 5, 7, 13]. More recent work introduced profit-aware performance measures and scalable learners that better align predictive outputs with financial outcomes [12, 16].

In parallel, large-scale data systems research developed practical stream and dataflow models that support fault-tolerant, low-latency processing over unbounded event streams [14, 15]. These foundations are directly relevant to ACIS-X because collections intelligence depends on continuous processing of customer, invoice, and payment events where timeliness and correctness must co-exist. In other words, the platform challenge is not only model selection but end-to-end decision semantics under real operational timing

constraints.

This chapter therefore treats literature review as an architecture-design activity. Instead of listing papers in isolation, it extracts reusable principles for ACIS-X across five decision dimensions:

- **Predictive validity:** whether models separate high-risk and low-risk behaviors reliably.
- **Economic alignment:** whether evaluation reflects intervention value and treatment cost.
- **Operational explainability:** whether outputs can be interpreted and actioned by collections teams.
- **Runtime resilience:** whether decision pipelines remain correct under delay, disorder, and failure.
- **Lifecycle adaptability:** whether models and policies can be recalibrated under changing data regimes.

Therefore, this chapter presents a focused literature survey on up to sixteen highly relevant papers and frameworks, then synthesizes their implications for ACIS-X architecture, risk-policy design, and operations. The goal is to identify what has been solved, what remains unresolved, and how a modern event-driven collections platform should be structured to close these gaps.

In addition to reviewing model accuracy, this survey emphasizes production success criteria that are commonly under-specified in standalone research: (i) operational robustness under delayed or incomplete events, (ii) explainability of risk signals at decision time, (iii) measurable linkage between model output and collection outcome, (iv) controllability of policy thresholds, and (v) observability of action quality after deployment. This framing is essential because ACIS-X is intended for continuous decision operations, not only offline experimentation.

The outcome of this section is not merely a literature summary; it is a traceable evidence base for Chapter 3 onward, where requirements, planning, and implementation choices need to be justified by both algorithmic and systems-level findings.

2.2 Objective of Literature Survey

The objectives of this literature survey are:

- i. To study the progression of consumer credit-scoring methods from classical statistical techniques to modern machine-learning approaches and identify their relevance to receivables intelligence.
- ii. To analyze how recent research handles practical challenges such as class imbalance, probability calibration, model interpretability, and business-value alignment.
- iii. To examine benchmark studies and understand why model conclusions vary across datasets, domains, and evaluation criteria.
- iv. To investigate stream-processing and dataflow literature for scalability, fault tolerance, event-time correctness, and low-latency operational decisioning.
- v. To derive architecture-level insights that can be directly mapped to ACIS-X, including modular agent orchestration, resilient runtime behavior, and auditable action workflows.
- vi. To identify limitations in existing systems and frame a problem statement and solution direction that is realistic for production collections operations.
- vii. To compare how papers evaluate performance (AUC, sensitivity, precision, profitability) and identify which metrics are most meaningful for collections intervention design.
- viii. To extract implementation lessons related to deployment feasibility, model refresh cadence, and integration with monitoring and policy systems.
- ix. To evaluate how each reviewed method supports account-level prioritization and treatment sequencing in practical collections teams.
- x. To analyze tradeoffs between model complexity and deployment constraints such as inference latency, monitoring overhead, and operational maintainability.
- xi. To identify how literature addresses delayed, out-of-order, or corrected events and determine the implications for event-driven financial workflows.
- xii. To assess whether reported methods provide sufficient explanation artifacts (feature attributions, reason codes, policy context) for audit and compliance usage.
- xiii. To investigate the extent to which current research supports closed-loop learning, where action outcomes feed back into periodic model and policy updates.

- xiv. To derive measurable criteria for ACIS-X evaluation beyond predictive discrimination, including action timeliness, intervention quality, and governance traceability.
- xv. To establish a structured bridge from literature evidence to system requirements, ensuring that architecture choices in later chapters are defensible and reproducible.

These objectives are intentionally application-driven: the literature is used as evidence for concrete design choices in ACIS-X rather than as an isolated academic survey.

Collectively, these objectives ensure that the survey informs both *what to build* (modeling and systems capabilities) and *how to operate it* (monitoring, governance, and continuous improvement). This dual focus is necessary for an intelligent collections platform intended for sustained real-world use.

2.3 Summary of Literature Survey

The literature survey on event-driven collections intelligence reveals three core knowledge pillars: credit-scoring methodology, business-aware model evaluation, and low-latency stream execution. The reviewed papers collectively show that reliable receivables decisioning depends on both predictive performance and operational delivery. The following summaries present the key contributions and their direct relevance to ACIS-X.

To maintain methodological consistency, each paper summary below captures contribution, practical value, and limitations. This allows a clearer mapping from research evidence to architecture decisions.

[1] Event-Driven Design for Agents and Multi-Agent Systems

AUTHOR: Sean Falconer

This Confluent whitepaper presents practical guidance for designing agent-based systems using an event-driven backbone. It explains how asynchronous event streams improve decoupling between agents, support independent scaling, and enable reliable coordination across distributed components. For ACIS-X, this directly supports the use of Kafka topics as the communication and orchestration layer for specialized agents.

A further contribution is the emphasis on production-readiness concerns such as observability, replayability, and fault isolation in multi-agent workflows. These ideas align with ACIS-X design choices around durable event contracts, retry-safe processing, and runtime resilience under variable load.

[2] Statistical Classification Methods in Consumer Credit Scoring: A Review

AUTHORS: D. J. Hand and W. E. Henley

This foundational review explains the statistical basis of consumer credit scoring and highlights practical deployment concerns that are often overlooked in pure algorithmic studies. It discusses model discrimination, calibration context, class imbalance, and the consequences of reject populations. The paper remains highly relevant because it frames scoring as an operational decision process rather than a purely mathematical exercise. For ACIS-X, this supports the need to combine model outputs with policy governance and action traceability.

An additional insight is that statistical validity can degrade when deployment conditions differ from training assumptions. ACIS-X addresses this by treating model output as one stage in a monitored pipeline rather than as a final isolated truth.

[3] A Survey of Credit and Behavioural Scoring

AUTHORS: Lyn C. Thomas

This survey distinguishes application scoring from behavioral scoring and demonstrates how risk prediction evolves across the customer lifecycle. It emphasizes that dynamic performance monitoring and repeated re-estimation are essential in lending environments. The work is directly aligned with ACIS-X, where risk signals are continuously updated based on new invoices, payments, and account events. It motivates lifecycle-aware state tracking rather than one-time score generation.

The paper also implies that temporal context should be first-class in model design. In ACIS-X, this translates to stateful event aggregation and time-sensitive features, especially for accounts that oscillate between normal and overdue behavior.

[4] Benchmarking State-of-the-Art Classification Algorithms for Credit Scoring

AUTHORS: B. Baesens, T. Van Gestel, S. Viaene, M. Stepanova, J. Suykens, and J. Vanthienen

This benchmark paper compares major classification approaches under controlled credit-scoring settings and demonstrates that model superiority depends on data conditions and evaluation design. The central contribution is methodological rigor in comparing algorithms beyond isolated case studies. It provides evidence that model selection should be context-dependent and repeatedly validated. ACIS-X benefits from this insight by supporting modular scoring components that can be replaced or rebenchmarked over time.

Its practical limitation is that benchmark conclusions are still tied to sampled datasets

and historical periods. ACIS-X mitigates this risk through periodic re-evaluation loops and model abstraction at the service boundary.

[5] Recent Developments in Consumer Credit Risk Assessment

AUTHORS: Jonathan N. Crook, David B. Edelman, and Lyn C. Thomas

This work synthesizes developments in consumer credit-risk assessment and reviews important operational themes such as validation quality, borrower heterogeneity, and decision strategy adaptation. It highlights that real-world collections outcomes are sensitive to both model quality and treatment policy. This directly informs ACIS-X design where predictive and action stages are separated but tightly linked through structured events.

The key implication for ACIS-X is that policy effectiveness must be measured jointly with model outputs. This motivates event-level traceability between prediction publication and collections action outcomes.

[6] Support Vector Machines for Credit Scoring and Discovery of Significant Features

AUTHORS: Tony Bellotti and Jonathan Crook

The paper evaluates SVM-based credit scoring and investigates feature significance to improve interpretability alongside discrimination power. It shows how non-linear methods can improve classification in structured financial datasets while still enabling feature-level understanding. For ACIS-X, this supports integrating explainable predictors where risk reasons can be propagated to downstream collection actions.

However, model explainability remains sensitive to feature engineering quality. ACIS-X therefore benefits from standardized feature lineage and reason-code publishing in risk events.

[7] Consumer Credit-Risk Models via Machine-Learning Algorithms

AUTHORS: Amir E. Khandani, Adlar J. Kim, and Andrew W. Lo

This study demonstrates practical gains from machine-learning approaches in consumer credit-risk prediction at large scale. The key implication is that rich feature interactions can materially improve predictive discrimination compared to simpler baselines. ACIS-X can leverage this principle through richer event-derived features at invoice and customer levels, provided model governance remains strong.

The paper also reinforces the value of data breadth. For ACIS-X, this suggests sustained enrichment and data quality controls are as important as algorithm choice.

[8] Multiple Classifier Architectures and Their Application to Credit Risk Assessment**AUTHORS:** Steven Finlay

This paper focuses on architecture-level combinations of classifiers and shows that multi-model designs can improve performance stability across scenarios. It argues that how models are composed can matter as much as which single algorithm is chosen. The finding is aligned with ACIS-X, where independent agent stages can support ensemble-style risk workflows and staged decision refinement.

Its relevance increases in volatile environments where single-model brittleness is common. ACIS-X can operationalize this through configurable model orchestration and fallback decision paths.

[9] A Comparative Assessment of Ensemble Learning for Credit Scoring**AUTHORS:** Gang Wang, Jinxing Hao, Jian Ma, and Hongbing Jiang

This comparative study evaluates ensemble methods in credit-scoring contexts and reports robust improvements under many data settings. The paper highlights variance reduction and better generalization as practical advantages of ensemble learning. For ACIS-X, this supports designing model-serving interfaces that are ensemble-compatible instead of tightly coupled to one learner.

The tradeoff is additional computational and governance complexity. ACIS-X should therefore pair ensemble usage with clear monitoring thresholds and explainability constraints.

[10] An Experimental Comparison of Classification Algorithms for Imbalanced Credit Scoring Data Sets**AUTHORS:** Iain Brown and Christophe Mues

The study focuses on the imbalanced nature of credit datasets where adverse outcomes are rare but high-impact. It compares algorithm behavior under skewed classes and demonstrates that imbalance treatment strongly affects operational usefulness. This is directly relevant to ACIS-X, where high-risk delinquency classes are sparse and must still be prioritized correctly for collections action.

This work supports threshold strategies that optimize intervention efficiency rather than overall accuracy alone. ACIS-X can embed this logic in risk-to-action policy mappings. By explicitly tuning these thresholds, the system minimizes false positives and prevents

unnecessary outreach to reliable customers.

[11] Consumer Credit Risk: Individual Probability Estimates Using Machine Learning

AUTHORS: Jochen Kruppa, Alexandra Schwarz, Gerhard Arminger, and Andreas Ziegler

This paper addresses individualized probability estimation and emphasizes the quality of account-level risk outputs. The contribution is important for collections prioritization because interventions are issued per customer or invoice, not only by portfolio segment. ACIS-X adopts the same orientation by emitting structured account-level risk events to downstream agents.

The paper also indicates that calibration quality matters for operational confidence. In ACIS-X, calibrated probabilities can improve prioritization fairness and escalation consistency.

[12] Development and Application of Consumer Credit Scoring Models Using Profit-Based Classification Measures

AUTHORS: Thomas Verbraken, Cristian Bravo, Richard Weber, and Bart Baesens

This study introduces profit-aware classification measures and demonstrates why AUC-only evaluation can be misaligned with business decision value. It provides a practical framework for linking predictive decisions to economic outcomes. ACIS-X directly benefits from this perspective by aligning risk thresholds and action priorities with recoverability and intervention value.

Its practical strength is decision realism: interventions incur costs, and delayed action changes expected recovery. ACIS-X can operationalize this through policy objectives tied to outcome economics.

[13] Benchmarking State-of-the-Art Classification Algorithms for Credit Scoring: An Update of Research

AUTHORS: Stefan Lessmann, Bart Baesens, Hsin-Vonn Seow, and Lyn C. Thomas

This update strengthens benchmarking practice by using broader datasets and more rigorous statistical comparisons. It reinforces that performance claims must be interpreted cautiously across changing data regimes. For ACIS-X, the implication is clear: model performance should be continuously monitored and periodically revalidated under production drift.

It also underscores reproducibility discipline, which ACIS-X can support via versioned model metadata and consistent event schema contracts.

[14] Discretized Streams: Fault-Tolerant Streaming Computation at Scale

AUTHORS: Matei Zaharia, Tathagata Das, Haoyuan Li, Timothy Hunter, Scott Shenker, and Ion Stoica

This work proposes a practical stream model for deterministic, fault-tolerant low-latency computation. Its main relevance is architectural: scalable stream intelligence must be designed with recovery semantics from the outset. ACIS-X follows this direction by separating event producers and consumers across robust messaging layers and replay-aware processing logic.

The core lesson is that latency without deterministic recovery is insufficient for mission-critical workflows. ACIS-X should preserve this balance through robust retry and replay patterns.

[15] The Dataflow Model: Balancing Correctness, Latency, and Cost

AUTHORS: Tyler Akidau, Robert Bradshaw, Craig Chambers, Slava Chernyak, Rafael J. Fernandez-Moctezuma, Reuven Lax, Sam McVeety, Daniel Mills, Frances Perry, Eric Schmidt, and Sam Whittle

The Dataflow model formalizes event-time semantics, handling of late data, and tradeoffs among correctness, latency, and cost. It is especially relevant for financial operations where delayed records and revisions are common. ACIS-X depends on these concepts to keep risk and action outputs consistent even when upstream events arrive out of order.

This is particularly important for receivables where settlement events may lag operational windows. ACIS-X can apply event-time aware logic to reduce inconsistent actioning and duplicate interventions.

[16] XGBoost: A Scalable Tree Boosting System

AUTHORS: Tianqi Chen and Carlos Guestrin

XGBoost introduced an efficient and scalable gradient-boosting framework that became a standard baseline for structured-data risk modeling. Its importance in collections intelligence is twofold: high predictive quality and practical train/inference efficiency. ACIS-X can incorporate this family of models in prediction stages while preserving explainability through feature-importance and reason-code interfaces.

Its engineering practicality makes it suitable for frequent retraining cycles. ACIS-X can exploit this for controlled model refresh under monitored drift.

2.3.1 Scalability and Real-Time Detection Challenges

Across the surveyed work, two recurring system-level challenges emerge for ACIS-X-like platforms:

- **Unbounded event handling with correctness guarantees:** Stream systems must process out-of-order and late data without losing decision consistency [14, 15].
- **Model quality under operational constraints:** Credit models must remain robust under class imbalance, drift, and changing business objectives while sustaining low-latency scoring [10, 13].

In production receivables operations, these challenges are tightly coupled. High latency reduces intervention value, while unstable model behavior reduces policy trust and action consistency. The literature therefore supports architectures that jointly optimize predictive quality, runtime resilience, and decision governance.

Another practical challenge is synchronization between state updates and action triggers. If risk recalculation and collections actioning are not coordinated, operational teams may receive stale or contradictory guidance. ACIS-X should mitigate this through explicit event contracts, idempotent action semantics, and bounded-latency observability signals.

A final concern is scaling governance alongside throughput. As event volume increases, auditability often degrades unless reason codes, model versions, and policy identifiers are embedded directly in events. This requirement is consistent across both credit-risk and stream-processing literature and should be treated as mandatory in ACIS-X.

Beyond these core concerns, scalability must be analyzed across four interacting layers:

- **Data-plane scalability:** the ingestion layer must sustain bursty invoice and payment traffic without message loss, consumer starvation, or queue amplification effects.
- **State-plane scalability:** customer-level aggregates and risk features must be updated under concurrent event flows while preserving deterministic read-after-write behavior for downstream actions.
- **Model-plane scalability:** scoring services must handle concurrent inference requests with bounded latency, while supporting safe model rollouts, shadow evaluations, and

rollback options.

- **Operations-plane scalability:** alerting, dashboards, and remediation pipelines must remain actionable for human teams as account volume grows, avoiding alert fatigue and intervention backlog.

Real-time constraints in collections systems are not only computational; they are economic. A delayed risk signal may still be statistically correct but operationally suboptimal if intervention windows have narrowed. Consequently, response-time objectives should be framed in business terms, such as time-to-action for high-risk accounts and time-to-reconciliation for corrected events.

Another important challenge is temporal inconsistency between event sources. Payment confirmations, settlement corrections, and external enrichment updates may arrive with different clocks and reliability profiles. Without strict event-time and processing-time separation, systems can trigger premature escalations or duplicate actions. Literature on stream correctness [15] indicates that explicit watermarking, late-arrival handling, and replay-safe processing are essential for dependable financial decisioning.

Finally, scaling model quality under drift remains difficult. Portfolio behavior changes with macroeconomic shocks, policy changes, and customer mix shifts. Systems that scale throughput but ignore drift governance often accumulate silent model degradation. This reinforces the need for continuous monitoring of calibration, class-balance shift, and action-outcome alignment at production cadence.

2.4 Drawbacks of Existing Systems

Existing literature and industrial practice reveal common limitations:

- **Metric-business mismatch:** Many implementations prioritize discrimination metrics without explicitly optimizing intervention economics or recoverability value [12].
- **Benchmark fragility:** Model superiority is often data-specific and may degrade under portfolio shifts, market changes, or different product segments [4, 13].
- **Weak lifecycle governance:** Continuous recalibration, drift monitoring, and controlled model updates are not consistently addressed in operational deployments.
- **Limited orchestration-awareness:** Predictive systems are frequently developed independently of runtime guarantees such as fault recovery and event-time consis-

tency [15].

- **Explainability gaps at action level:** High-performing models are not always paired with transparent reason codes suitable for collections teams and compliance review.
- **Fragmented architecture:** Many systems still separate data ingestion, scoring, and action execution in ways that hinder auditability and closed-loop optimization.

Taken together, these drawbacks indicate that many systems optimize isolated components while under-optimizing the full decision lifecycle. ACIS-X must therefore be evaluated end-to-end: from event ingestion fidelity to measurable action outcome quality.

Additional drawbacks observed in real deployments further reinforce these gaps:

- **Weak event contract discipline:** loosely defined payload schemas produce downstream ambiguity, brittle parsing logic, and inconsistent cross-agent interpretation.
- **Insufficient idempotency controls:** duplicate message handling is often incomplete, causing repeated collection actions and customer-experience degradation.
- **Limited replay governance:** many systems support replay technically, but not operationally, leading to confusion about which actions should or should not be re-issued.
- **Sparse post-action analytics:** outcomes of reminders, escalations, and settlement plans are not systematically linked back to the originating risk decisions.
- **Inconsistent human override integration:** manual interventions are frequently captured outside the decision pipeline, reducing traceability and learning quality.
- **Underdeveloped policy simulation:** threshold and strategy changes are deployed without robust pre-deployment impact simulation on collections workload and recovery.

In aggregate, these shortcomings create a compounding effect: poor schema discipline undermines explainability, weak explainability undermines policy confidence, and low policy confidence leads to fragmented manual overrides that reduce model-learning signal quality. A modern system must address these dependencies explicitly.

2.5 Problem Statement

Organizations need a collections intelligence platform that continuously transforms heterogeneous financial events into timely, explainable, and value-aware actions. Exist-

ing approaches typically separate model development, event processing, and operational collections execution. This separation causes delayed intervention, inconsistent decision rationale, and weak accountability when outcomes are audited.

The central problem is to design a unified architecture that jointly satisfies:

- real-time event processing at scale,
- accurate and calibrated account-level risk estimation,
- policy-consistent and economically meaningful action selection,
- operational resilience with auditable decision trails.

The literature indicates that solving only one of these dimensions is insufficient. A production-ready solution must integrate prediction, runtime semantics, and action governance into a single controllable pipeline.

In practical terms, the unresolved problem is not only predicting delinquency, but deciding who should be contacted, when, and with what treatment strategy, under bounded latency and strict traceability constraints. This is the operational gap ACIS-X is designed to close.

From a systems perspective, the problem can be decomposed into functional and non-functional gaps:

- **Functional gap:** risk estimation, policy interpretation, and action execution are not consistently chained through auditable events.
- **Temporal gap:** decision latency and event disorder handling are rarely treated as first-class requirements in collections workflows.
- **Governance gap:** model versions, decision reasons, and policy parameters are not always persisted in a form suitable for compliance and root-cause analysis.
- **Learning gap:** intervention outcomes are weakly integrated into periodic recalibration, limiting closed-loop improvement.

Therefore, the central challenge is to engineer a unified platform where analytics, orchestration, and operational actioning are jointly optimized. The system must deliver low-latency decisions, maintain correctness under disorder and failure, support policy-level explainability, and continuously improve using measured outcomes.

2.6 Proposed Solution

To address the identified limitations and align with the literature-backed requirements, the proposed ACIS-X direction follows a layered and event-driven design. The platform unifies ingestion, state construction, risk estimation, policy interpretation, and action publication under a traceable workflow.

The design intentionally separates responsibilities by agent while preserving strong contracts between stages. This improves maintainability, supports model experimentation, and reduces failure propagation across the pipeline.

The proposed solution is guided by six design principles:

- **Event-first orchestration:** all significant state transitions and decisions are represented as explicit events.
- **Deterministic state reconstruction:** the platform can reproduce account state and decisions from event history for audit and replay.
- **Policy-aware modeling:** model outputs are treated as policy inputs, not standalone decisions.
- **Explainability by construction:** decision payloads include reason context, model identity, and policy versioning.
- **Operational resilience:** retries, monitoring, and self-healing are integrated in the baseline architecture rather than added later.
- **Closed-loop improvement:** action outcomes are fed back into periodic model and policy recalibration cycles.

Under this approach, ACIS-X is not a single model deployment; it is a continuous decision system with explicit lifecycle controls.

2.6.1 Event and State Intelligence Layer

The first layer continuously consumes customer, invoice, and payment events and converts them into standardized operational state. This includes behavior metrics, exposure aggregation, and enrichment joins from external context signals. The architecture follows stream/dataflow principles for low-latency and correctness-aware processing over unbounded events [14, 15].

Implementation priorities in this layer include schema validation, deduplication safeguards, event timestamp normalization, and deterministic aggregation windows. These controls are necessary to ensure downstream models receive coherent and trustworthy features.

To make this layer robust in production, ACIS-X should define explicit data contracts per event topic, including required fields, semantic constraints, and version compatibility rules. Contract checks should run at ingress and before inter-agent publication to prevent silent schema drift.

State construction in this layer should support both real-time access and reproducibility. That means maintaining materialized operational views for fast decisioning while preserving append-only event lineage for audit and replay. Feature derivation logic should be versioned so that historical decisions can be interpreted against the exact transformation rules active at decision time.

This layer should also maintain data quality indicators such as missing-field rates, delayed-event frequency, and source-specific reliability scores. These indicators can be propagated downstream as confidence modifiers for risk and action modules.

Suggested layer outputs include:

- normalized account-state events,
- derived behavior metrics with timestamps,
- enrichment-complete feature snapshots,
- data-quality and freshness metadata.

2.6.2 Risk and Policy Intelligence Layer

The second layer applies benchmarked, imbalance-aware predictive models and produces account-level risk probabilities with structured reasons. Model evaluation is aligned with both classification quality and business impact, including profit-aware decision framing where appropriate [11, 12, 13]. This ensures outputs are suitable for operational prioritization rather than static reporting.

This layer also supports policy-aware thresholding, calibration checks, and versioned model metadata. Together, these features enable safe model updates and clearer interpretation when outcomes are reviewed by operations or governance teams.

In practice, this layer should combine statistical predictions with configurable policy rules to produce final operational decisions. Rules may encode regulatory boundaries, customer treatment constraints, or business-priority conditions that cannot be left to model inference alone. A hybrid decision strategy improves controllability without discarding predictive signal quality.

Model management in this layer should include champion-challenger evaluation, drift-triggered retraining thresholds, and shadow deployment for safe model validation under live traffic. Calibration monitoring should be performed at segment level to prevent localized degradation that may be hidden by aggregate metrics.

The policy engine should support scenario simulation before threshold updates are promoted. For example, teams should be able to estimate expected increase in high-priority accounts and projected intervention workload under candidate policy changes.

Suggested layer outputs include:

- account-level risk probabilities and risk bands,
- reason-coded decision context,
- applied policy identifiers and threshold versions,
- model identity, calibration status, and confidence indicators.

2.6.3 Real-Time Actioning and Monitoring Layer

The third layer translates risk outcomes into collections actions with policy constraints, duplicate prevention, and escalation logic. Runtime monitoring, retry controls, and self-healing mechanisms are integrated to preserve service continuity under failures. Standardized event schemas and explainable decision payloads provide the governance trail required for compliance and continuous improvement.

Operationally, this layer should expose measurable service indicators such as action latency, escalation rate, retry frequency, and policy override frequency. These metrics form a feedback loop for both process improvement and model governance.

Action orchestration should implement explicit priority queues and channel strategies (for example, reminder, negotiation, escalation) based on risk band, account context, and policy constraints. Duplicate suppression must operate across both event idempotency keys and business keys to prevent repeated contact attempts from replayed or retried events.

Monitoring should be multi-dimensional: system health (lag, throughput, failure rate), decision health (distribution shifts, reason-code drift), and business health (promise-to-pay conversion, recovery rate, escalation fallout). This allows teams to distinguish infrastructure incidents from policy or model issues.

Self-healing behavior should include bounded retries, dead-letter routing, automated dependency checks, and controlled degradation modes. For example, if enrichment is temporarily unavailable, the system may fall back to conservative policy thresholds while flagging reduced-confidence decisions.

To operationalize continuous improvement, this layer should publish structured outcome events after each intervention stage. These outcomes form the training and policy feedback dataset for periodic optimization.

Under this design, ACIS-X provides four concrete advantages relative to fragmented alternatives:

- **Timeliness:** event-driven processing reduces delay between risk signal emergence and action publication.
- **Decision quality:** model evaluation is coupled with economic utility and class-imbalance sensitivity.
- **Operational confidence:** runtime health and failure recovery are explicit architectural concerns.
- **Governance readiness:** reason-coded outputs and structured event histories support audit and improvement cycles.

Beyond these advantages, the architecture supports continuous learning from outcomes, closing the loop between analytics and operational execution. By establishing this unified, event-driven foundation, the proposed solution directly addresses the operational gaps—such as decision latency and weak lifecycle governance—identified in existing systems. This architectural paradigm essentially restructures how both functional capabilities and non-functional constraints are formulated, ensuring that subsequent design phases are deeply grounded in practical deployment realities.

CHAPTER 3

REQUIREMENT ENGINEERING

Requirement engineering for ACIS-X is derived from implementation artifacts, architecture specifications, operational guides, test assets, and available report PDFs in the project workspace. The chapter consolidates requirements from the runtime bootstrap, topic management, schema contracts, deployment constraints, and validation strategy so the platform can be implemented and operated as a consistent decision system.

Primary evidence inputs for this chapter include:

- core architecture and flow documentation (README, ARCHITECTURE, ARCHITECTURE_MERMAID),
- runtime and configuration sources (run_acis.py, runtime/*, config/settings.py),
- quality and operations guidance (TESTING.md, DEPLOYMENT_GUIDE.md, pytest.ini, test suites),
- technical dependency manifest (requirements.txt),
- report assets in PDF form available under report-assets (metadata and project context validation).

3.1 Software and Hardware Tools Used

3.1.1 Software Requirements

Table 3.1 maps the software tools required for building the platform. ACIS-X uses a Python-centric stack with Kafka for event transport, SQLite for persistence, and pytest for verification. The selected tools prioritize fast iteration, explicit event contracts, and practical production transition.

Table 3.1 Software Requirements

Operating System	Windows / macOS / Linux
Development Environment	Visual Studio Code
Programming Language	Python 3.9+
Event Broker	Apache Kafka
Database	SQLite

3.1.2 Hardware Requirements

Table 3.2 maps the minimum and recommended hardware specifications. The current ACIS-X implementation supports local and production-like deployment using lightweight infrastructure.

Table 3.2 Hardware Requirements

Processor	Minimum 1 GHz; Recommended 2 GHz or more
Memory (RAM)	Minimum 1 GB; Recommended 4 GB and above
Storage	Minimum 20 GB free disk space

The baseline above is aligned with stated system targets in deployment documentation, including startup under 30 seconds, stable memory behavior under load, and sustained event throughput expectations.

3.2 Conceptual/Analysis Modeling

3.2.1 Use Case Diagram

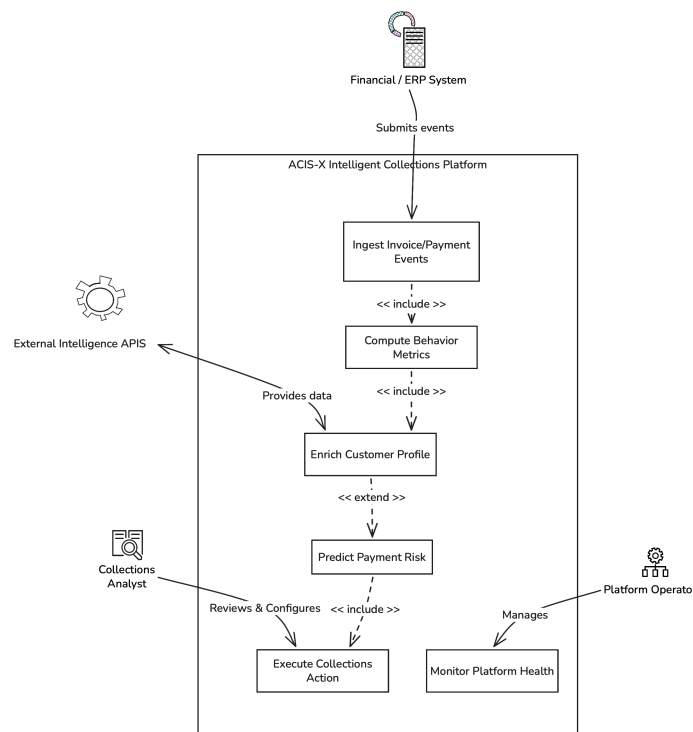


Fig. 3.1 Use case diagram for ACIS-X

Fig 3.1 maps the interactions between various actors and the ACIS-X system and separates external data and human supervision from its internal workflows during runtime.

The Use Case diagram identifies the primary actors (such as event producers, the orchestration runtime, external enrichment services, and collections managers) interacting with the core usage scenarios of the system.

3.2.2 Sequence Diagram

Fig 3.2 visualizes the asynchronous, topic-based communication flowing between dedicated agents (e.g., ScenarioGenerator, ExternalData, RiskScoring) and their centralized exchange via the Kafka Event Bus. It maps the temporal message exchanges between these agents.

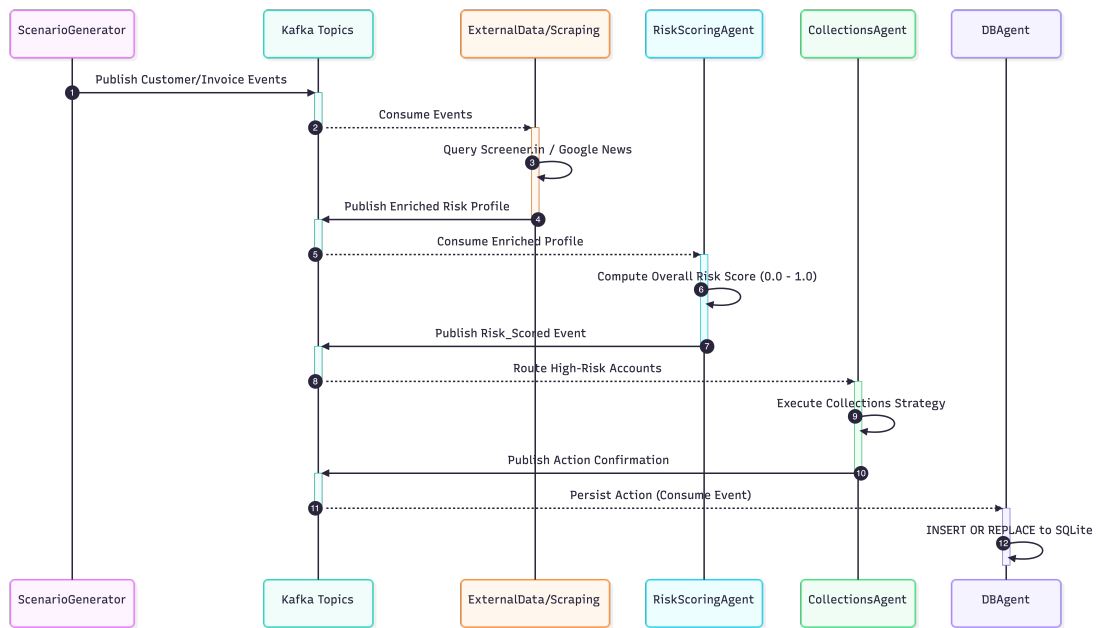


Fig. 3.2 Sequence diagram illustrating event communication flows

As depicted in the sequence diagram above, the event-driven architecture heavily relies on asynchronous message passing to ensure robust separation of concerns. Rather than invoking services through direct synchronous calls, which can lead to cascading failures and tight coupling, agents publish their state changes and analytical results directly to designated Kafka topics. For instance, the Scenario Generator emits initial simulation events that act as the catalyst for the workflow. These raw events are subsequently consumed and augmented by the External Data agent, which appends crucial market or litigation context.

Furthermore, this sequence of topic-based publications protects the system from cascading delays. If the Risk Scoring component goes offline momentarily, the upstream events simply remain intact within their respective Kafka partitions until the agent recovers.

3.2.3 Activity Diagram

Fig 3.3 details the step-by-step workflow for the data processing pipeline starting from raw invoice ingestion, progressing to metrics computation and enrichment, and culminating in collections escalations.

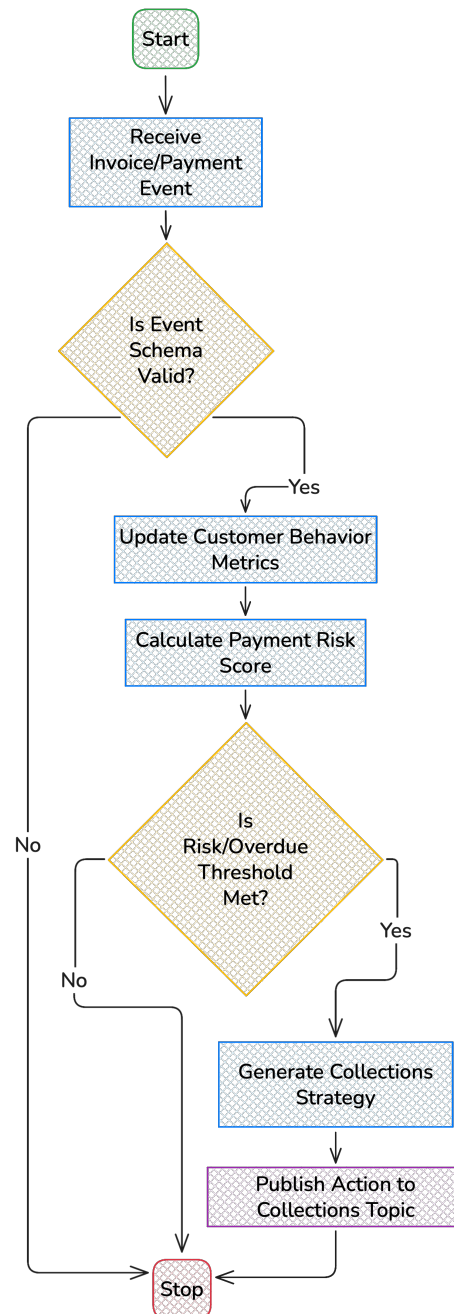


Fig. 3.3 Activity diagram mapping out the decision pipeline

As an invoice progresses through its lifecycle, the workflow dynamically adjusts its trajectory based on ongoing risk scoring, effectively deciding in real-time whether to pause for more data, alert an operator, or immediately escalate to collections.

3.2.4 State Chart Diagram

The State Chart Diagram maps the lifecycle of an account's state within ACIS-X, transitioning between normal operations, an early warning status during overdue detection, and a high-risk state triggering immediate policy/collections interventions. Fig 3.4 maps the valid state transitions an account can undergo.

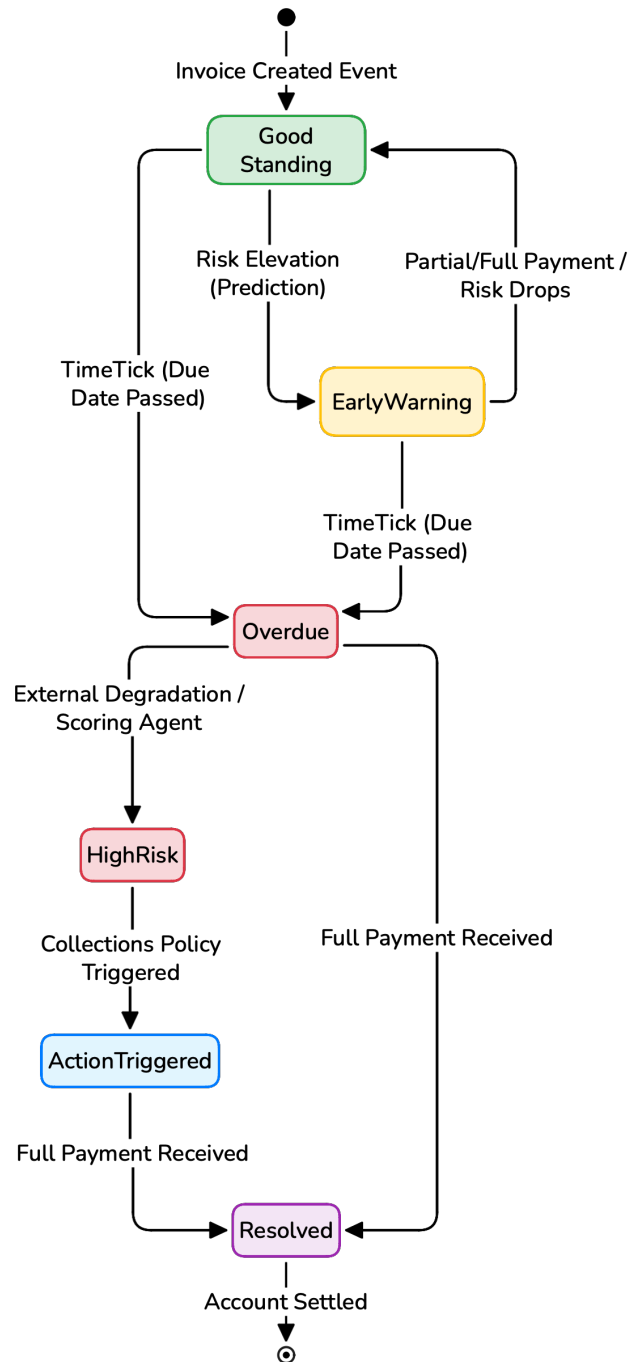


Fig. 3.4 State chart diagram specifying account state transitions

This explicit modeling of states prevents the system from generating redundant or

oscillating collection actions. An account must cross specific risk thresholds and satisfy temporal policy criteria before transitioning into a higher escalation tier, thereby ensuring that automated interventions remain economically justified and compliant with customer treatment guidelines.

3.3 Software Requirement Specification

This defines what ACIS-X must do and how well it must do it under practical conditions. Requirements are grouped into functional, non-functional, interface/data, and governance-oriented validation requirements. These specifications serve as the baseline against which system implementation and subsequent integration tests are evaluated.

3.3.1 Functional Requirements

- i. Event Ingestion:** The system must be able to continuously consume customer, invoice, and payment events from Kafka topics.
- ii. State Computation:** The system must compute and maintain real-time customer behavior metrics and exposure aggregations.
- iii. External Enrichment:** The system should fetch and integrate external financial and litigation data to enrich customer profiles.
- iv. Risk Prediction:** It must generate structured invoice-level payment-risk predictions based on unified account data.
- v. Action Routing:** The system must evaluate risk scores against policies and trigger appropriate collections actions automatically.

3.3.2 Non-Functional Requirements

- i. Performance:** The platform should process events efficiently with high throughput and minimal latency.
- ii. Reliability:** The system must ensure continuous operation with automated retry and dead-letter queue (DLQ) mechanisms for fault tolerance.
- iii. Scalability:** The architecture should scale horizontally by allowing multiple agent replicas within canonical consumer groups.
- iv. Maintainability:** The modular multi-agent structure and explicit event contracts must support easy updates and independent component lifecycle management.
- v. Observability:** The system should emit continuous logs, heartbeats, and metrics to ensure clear diagnostic visibility.

CHAPTER 4

PROJECT PLANNING

4.1 Project Planning

Project planning is a critical phase for ACIS-X because the system combines event-driven engineering, risk intelligence, operational controls, and reporting workflows. To keep development predictable, planning was broken into implementation streams with clear ownership, weekly checkpoints, and validation gates. The project started approximately three weeks ago, and the backend-heavy architecture and data pipeline components are already underway, while frontend/dashboard work remains pending.

i. Requirement Consolidation and Scope Freezing:

- Mapped requirements from architecture docs, runtime code, tests, and deployment guidelines.
- Locked Chapter 3 SRS scope around functional and non-functional requirements aligned with the reference format.

ii. Architecture and Module Planning:

- Organized work by layers: ingestion/state, enrichment/prediction, scoring/actioning, and control-plane resilience.
- Defined development order to prioritize stable event flow before UX and reporting surfaces.

iii. Backend Implementation Track:

- Prioritized agent orchestration, topic contracts, persistence paths, and runtime startup/shutdown reliability.
- Focused first three weeks on core ACIS-X pipeline correctness and integration readiness.

iv. Data and External Integration Track:

- Planned integration of external financial and litigation signals with fallback mechanisms.
- Added checkpoints for API timeout behavior, retry strategy, and enrichment consistency.

v. Testing and Validation Track:

- Established unit-first validation using pytest markers and architecture-fix regres-

sion tests.

- Scheduled integration validation after backend stabilization and before frontend coupling.

vi. Performance and Reliability Milestones:

- Planned checkpoints for startup time, lag behavior, memory stability, and consumer-group correctness.
- Included self-healing and monitoring verification as mandatory pre-demonstration criteria.

vii. Frontend and Visualization Planning:

- Frontend dashboard/workbench was intentionally scheduled after backend API and event contracts matured.
- Current status: frontend implementation is pending and is the next major execution block.

viii. Documentation and Report Integration:

- Planned chapter-wise documentation in parallel with implementation to reduce end-cycle documentation risk.
- Ensured report structure mirrors reference chapter/subheading style while preserving ACIS-X specifics.

ix. Risk Management and Mitigation:

- Key risks: external API unreliability, schema drift, event lag spikes, and timeline compression for frontend completion.
- Mitigation: fallback data paths, strict event contracts, continuous testing, and phased frontend rollout.

x. Milestone Tracking and Delivery Governance:

- Weekly milestone tracking with completed/in-progress/pending status tags.
- Delivery sequencing follows: backend hardening -> frontend completion -> end-to-end validation -> final submission.

4.1.1 Scheduling

Table 4.1 shows the scheduling for ACIS-X is based on an 8-week execution window, with approximately the first 3 weeks already completed. The schedule emphasizes backend-first completion, then frontend implementation, integration, and final validation/documentation.

Table 4.1 ACIS-X Project Schedule (8-Week Plan)

Week	Task	Status
Week 1	Literature alignment, architecture review, scope definition, environment setup.	Completed
Week 2	Requirement engineering draft, chapter structuring, runtime/dependency mapping.	Completed
Week 3	Core backend pipeline refinement, table/LaTeX stabilization, chapter updates.	Completed
Week 4	Frontend planning finalization, UI wireframe selection, API binding contracts.	Completed
Week 5	Frontend implementation (dashboards, risk/action views, status panels).	In Progress
Week 6	End-to-end integration: frontend with event and persistence outputs.	In Progress
Week 7	System validation, performance checks, bug fixes, UX improvements.	Pending
Week 8	Final report polishing, consolidated testing evidence, final submission prep.	Pending

The scheduled progression reflects a deliberate methodology to de-risk the most complex backend mechanics—such as event orchestration and state management—early in the lifecycle. By front-loading architecture refinement and pipeline stabilization in the initial weeks, the project guarantees that downstream dependencies do not inherit hidden architectural flaws. Moving forward, the critical focus transitions into the ongoing end-to-end integration and the impending system validation stages. These final weeks are instrumental for fine-tuning predictive latencies, ironing out frontend component linkages, and comprehensively evaluating whether the resulting prototype successfully fulfills the overarching resilience and scalability goals of intelligent collections.

CHAPTER 5

SYSTEM DESIGN

5.1 System Architecture

The system architecture of ACIS-X is designed as an event-driven, multi-agent platform for intelligent accounts receivable monitoring and collections decision support. The architecture separates data ingestion, customer-state construction, external enrichment, prediction, risk scoring, collections actioning, persistence, and runtime supervision into independent but coordinated modules.

At the center of the architecture is a Kafka-based event backbone. Each major business event, such as customer creation, invoice generation, payment posting, metric computation, risk scoring, and collections action, is published to a dedicated topic. This allows agents to operate independently while still participating in a unified processing workflow. The event-based design also improves traceability because each decision can be connected back to the events and derived signals that produced it.

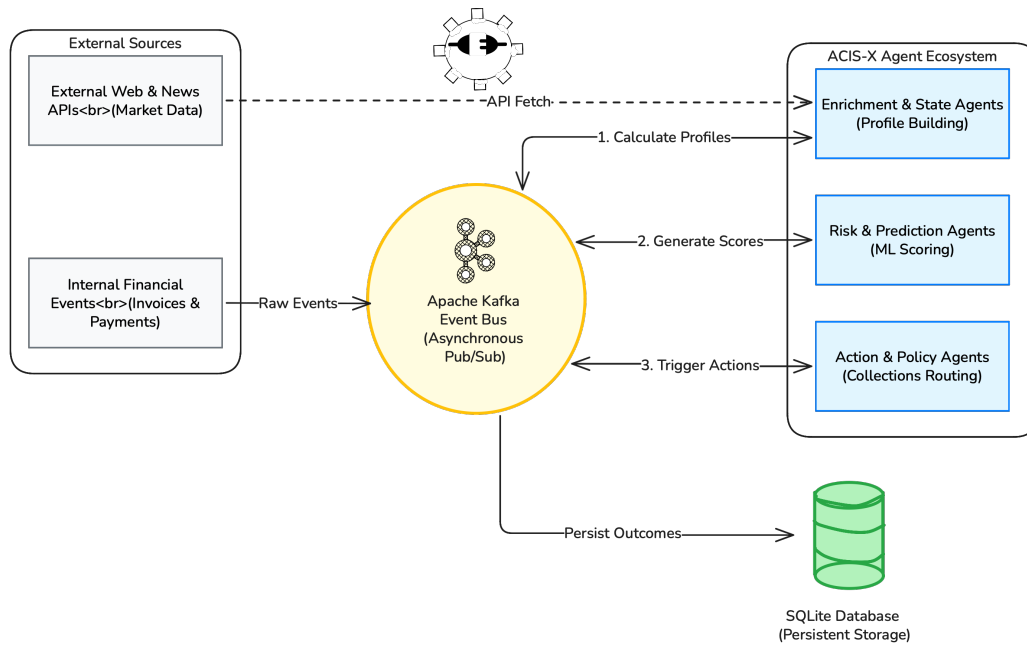


Fig. 5.1 System architecture of ACIS-X

Fig 5.1 maps the five major architectural layers of ACIS-X. The input layer provides customer, invoice, payment, and external source information. The event bus layer routes this information through Kafka topics. The agent layer performs state computation, enrichment,

aggregation, prediction, risk scoring, and collections actioning. The storage and query layer persists events and derived records using SQLite, while QueryAgent and MemoryAgent provide structured access to state. The user and operations layer exposes the processed outputs through the FastAPI backend-for-frontend, React dashboard, monitoring services, registry services, and self-healing runtime modules.

The architecture is intentionally modular. Business agents focus on receivables intelligence, while runtime agents focus on operational reliability. This separation ensures that failures in monitoring, recovery, or placement logic do not directly alter the business decision rules, and business logic can evolve without rewriting the runtime orchestration layer.

5.2 Component Design / Module Decomposition

The component design of ACIS-X decomposes the platform into specialized agents and services. Each component has a clearly defined responsibility, communicates using event contracts, and produces outputs that can be consumed by downstream modules. This design supports maintainability, testing, and future extension of the platform.

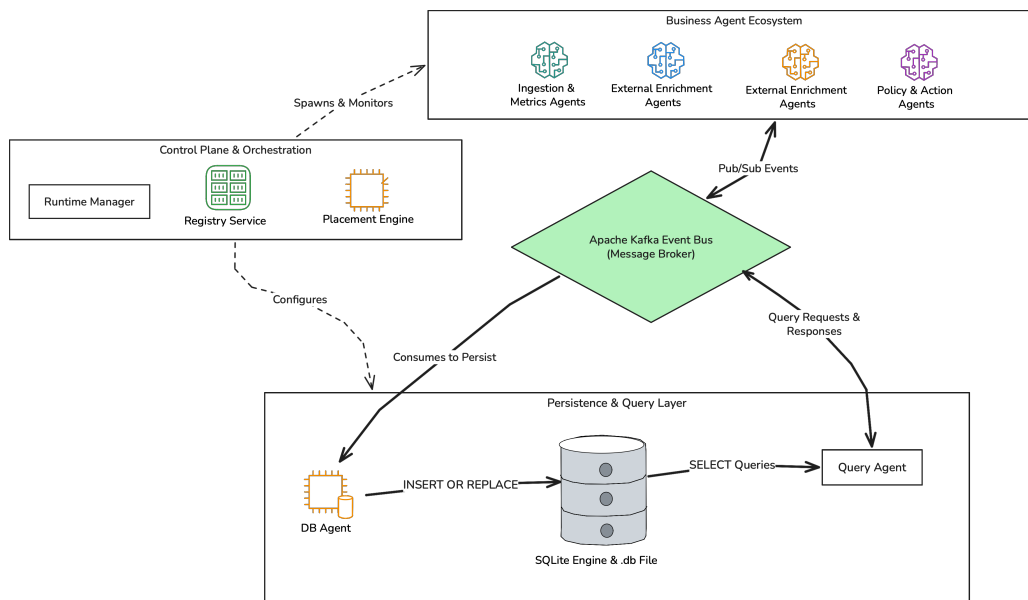


Fig. 5.2 Component design of ACIS-X

Fig 5.2 maps the main functional modules and internal components of the system. ScenarioGeneratorAgent produces simulated operational events. CustomerStateAgent converts invoice and payment behavior into customer metrics. ExternalDataAgent and ExternalScrapingAgent enrich customer context with external signals. AggregatorAgent

combines internal and external evidence into a broader customer profile. PaymentPredictionAgent estimates payment risk for open invoices, and RiskScoringAgent converts prediction and context into final risk decisions. CollectionsAgent uses those decisions to recommend appropriate collections actions.

The persistence and query components support the business agents. DBAgent stores events, customer records, invoices, payments, metrics, risk profiles, and collections actions. QueryAgent responds to structured read requests, while MemoryAgent maintains fast runtime context for frequently used customer state. Table 5.1 maps the responsibilities, inputs, and outputs of these core components. Operational components such as RegistryService, RuntimeManager, MonitoringAgent, PlacementEngine, SelfHealingAgent, and DLQMonitorAgent supervise agent health, recovery, placement, and failed-event handling.

Table 5.1 Core Components of ACIS-X

Component	Responsibility	Main Input	Main Output
ScenarioGenerator-Agent	Generates sample customer, invoice, and payment activity for simulation and validation.	Scenario settings	Customer, invoice, and payment events
CustomerStateAgent	Computes customer-level receivables metrics from invoice and payment behavior.	Invoice and payment events	Customer metrics
ExternalDataAgent	Collects structured external financial or company-level enrichment data.	Customer identity	External financial metrics
ExternalScrapingAgent	Extracts litigation, news, or web-based risk indicators related to customers.	Customer/company profile	External risk indicators
AggregatorAgent	Combines internal metrics and external indicators into a consolidated risk profile.	Metrics and enrichment data	Aggregated risk profile

Continued on next page

Table 5.1 *Continued from previous page*

Component	Responsibility	Main Input	Main Output
PaymentPrediction-Agent	Estimates payment delay or non-payment likelihood for open invoices.	Invoice and customer metrics	Payment prediction
RiskScoringAgent	Produces final risk scores and risk bands using prediction and context.	Prediction and profile data	Risk score event
CollectionsAgent	Converts risk outcomes into prioritized collections recommendations.	Risk score event	Collections action
DBAgent	Persists raw events and derived records into the local database.	Domain and system events	SQLite records
QueryAgent	Serves structured read requests for agents and the API layer.	Query requests	Query responses
MemoryAgent	Maintains fast runtime customer and decision context.	Domain events	Cached state
MonitoringAgent	Tracks runtime health, heartbeats, lag, and error conditions.	Health and system events	Health status
SelfHealingAgent	Initiates recovery actions when agents become unhealthy or fail.	Health status	Recovery action
RegistryService	Maintains active agent identity, status, and capability metadata.	Agent registration	Agent registry state

5.3 Interface Design

The interface design of ACIS-X defines how agents, storage, external sources, and the user-facing dashboard communicate with each other. Since the system is event-driven, most internal interfaces are asynchronous Kafka topics. Fig 5.3 maps the external and internal interfaces of the system. Each topic represents a specific type of event and allows producers and consumers to remain loosely coupled.

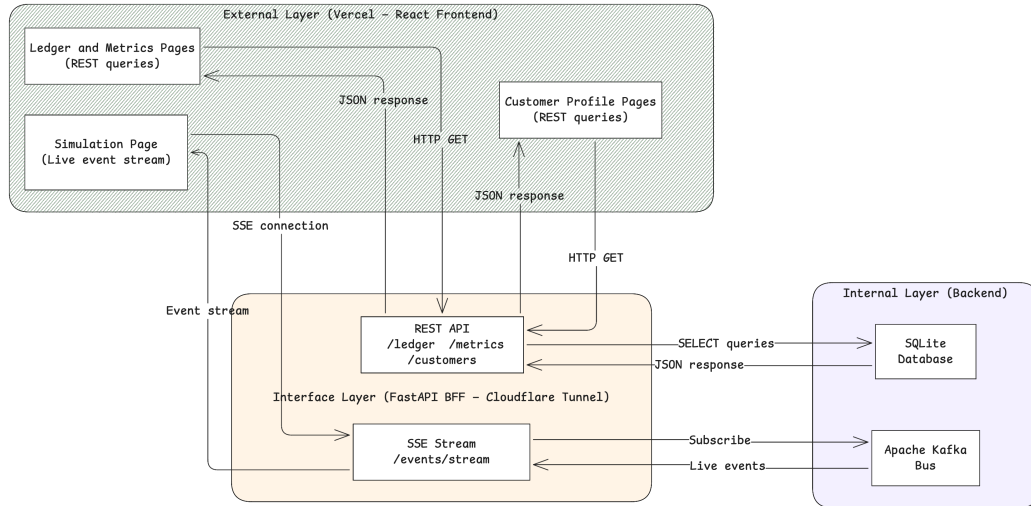


Fig. 5.3 Interface design of ACIS-X

ACIS-X uses four primary interface categories. Internal interfaces connect agents through Kafka topics. External interfaces allow enrichment agents to access financial, litigation, news, or company-related data. Persistence interfaces connect DBAgent and QueryAgent with the SQLite database. User-facing interfaces expose risk profiles, collections actions, metrics, and runtime status through the FastAPI backend and React dashboard.

The topic interface is the most important internal communication mechanism. It allows the system to process events incrementally and supports future scaling by adding more producers or consumers without changing the entire architecture. Table 5.2 maps the Kafka topics used for communication between various agents.

By standardizing these asynchronous boundaries, ACIS-X isolates failure domains and ensures that new analytical models, monitoring tools, or external integrations can be introduced seamlessly. This interface-driven decoupling is fundamental to maintaining a resilient and adaptable collections intelligence platform under evolving business requirements.

Table 5.2 Kafka Topic Interface Summary

Topic	Producer	Consumer	Purpose
acis.customers	ScenarioGenerator-Agent	DBAgent, MemoryAgent, risk/profile agents	Customer master events
acis.invoices	ScenarioGenerator-Agent	DBAgent, Customer-StateAgent, Payment-PredictionAgent	Invoice lifecycle data
acis.payments	ScenarioGenerator-Agent	DBAgent, Customer-StateAgent	Payment behavior data
acis.metrics	Customer-StateAgent, enrichment agents	AggregatorAgent, PaymentPredictionAgent, DBAgent	Derived customer metrics
acis.predictions	PaymentPrediction-Agent	RiskScoringAgent, DBAgent	Invoice-level prediction output
acis.risk	AggregatorAgent, RiskScoringAgent	CollectionsAgent, DBAgent, dashboard layer	Customer and invoice risk state
acis.collections	CollectionsAgent	DBAgent, dashboard layer	Collections recommendation output
acis.query.request	API layer or agents	QueryAgent	Structured read request
acis.query.response	QueryAgent	API layer or requesting agent	Structured query result
acis.agent.health	Runtime agents	MonitoringAgent, SelfHealingAgent	Agent health and heartbeat tracking
acis.dlq	Failed processors	DLQMonitorAgent	Failed-event inspection and recovery

5.4 Data Structure Design

The data structure design of ACIS-X is centered on structured events and persistent domain records. Events are used for communication between agents, while database records are used for durable storage and later query. This separation allows the system to process live data in real time while still preserving historical evidence for audit, replay, and report generation.

The event schema is designed to provide traceability. Each event contains an identifier, event type, source, timestamp, entity reference, and payload. Table 5.3 maps the primary data structures used across the ACIS-X platform. Derived structures such as customer metrics, risk profiles, and collections actions include reason fields so downstream modules can explain how a decision was produced.

Table 5.3 Data Structures Used in ACIS-X

Data Structure	Description
Event schema	Common message format containing event id, event type, source, timestamp, entity id, and payload.
Event envelope	Transport wrapper used for metadata, correlation, retry handling, and routing context.
Customer record	Stores customer identity, organization information, account status, and related metadata.
Invoice record	Stores invoice id, customer id, issue date, due date, amount, paid amount, and invoice status.
Payment record	Stores payment id, invoice id, customer id, payment date, amount, and settlement reference.
Customer metrics payload	Stores outstanding amount, overdue amount, average delay, payment ratio, and behavioral indicators.
External financial payload	Stores company-level enrichment values such as financial score, stability signal, and source metadata.
Litigation/news payload	Stores external legal or public-risk indicators collected for customer risk enrichment.

Continued on next page

Table 5.3 *Continued from previous page*

Data Structure	Description
Risk profile payload	Stores aggregated score, risk band, contributing factors, reason codes, and computed timestamp.
Collections action payload	Stores recommended action, priority, reason, cooldown key, and action status.
Agent registry record	Stores agent id, role, status, host, heartbeat time, and capability information.
Query request/response	Represents structured read requests and responses used by QueryAgent for database-backed access.

5.5 Algorithm Design

Algorithm design in ACIS-X focuses on converting event streams into operational decisions. Each algorithm is implemented through one or more agents, and each stage publishes structured output for the next stage. The overall process begins with ingestion and persistence, continues through customer-state computation and enrichment, and ends with risk scoring, collections actioning, and runtime recovery.

5.5.1 Event Ingestion and Persistence Algorithm

The event ingestion and persistence algorithm ensures that incoming business events are validated, stored, and made available to downstream agents.

- Receive customer, invoice, payment, metric, risk, or collections event from the relevant Kafka topic.
- Validate the event structure, mandatory fields, event type, and entity reference.
- Check the event identifier to reduce duplicate processing during retries or replay.
- Transform the event payload into the corresponding database record when persistence is required.
- Store the raw event and derived domain record through DBAgent.
- Publish or expose stored state for later access through QueryAgent and MemoryAgent.
- Route invalid or failed events to dead-letter handling for inspection and recovery.

5.5.2 Customer Metrics Computation Algorithm

The customer metrics computation algorithm converts invoice and payment history into useful receivables indicators.

- Consume invoice and payment events for a customer.
- Retrieve related invoice and payment history from local state or QueryAgent.
- Calculate outstanding balance, overdue balance, paid amount, and unpaid amount.
- Calculate behavioral features such as average payment delay, on-time payment ratio, and overdue frequency.
- Generate a customer metrics payload with computed values and timestamp.
- Publish the metrics event for aggregation, prediction, storage, and dashboard use.

5.5.3 External Enrichment and Risk Aggregation Algorithm

The external enrichment and aggregation algorithm expands customer evaluation beyond internal payment behavior.

- Identify customers requiring external enrichment using customer profile or metric events.
- Resolve the company or customer reference needed for external lookup.
- Collect financial, litigation, news, or web-based indicators through enrichment agents.
- Normalize external values into comparable risk signals.
- Combine internal receivables metrics and external risk indicators.
- Produce an aggregated customer risk profile with score, reasons, and evidence references.
- Publish the aggregated profile for prediction, risk scoring, persistence, and visualization.

5.5.4 Payment Prediction and Final Risk Scoring Algorithm

The prediction and risk scoring algorithm determines the likelihood and severity of payment risk for customers and invoices.

- Select active or open invoices that require risk evaluation.

- Read current customer metrics, invoice attributes, and aggregated external profile.
- Estimate payment delay or non-payment likelihood using available behavioral and contextual indicators.
- Generate a prediction event containing probability, confidence, and reason context.
- Refine the prediction using business rules, risk thresholds, and customer-level context.
- Assign the final risk band, such as low, medium, high, or critical.
- Publish the final risk score event for collections actioning, storage, and dashboard display.

5.5.5 Collections Action and Self-Healing Algorithm

The collections action algorithm converts final risk output into an operational recommendation, while the self-healing algorithm maintains runtime reliability.

- Consume high-risk or updated risk-score events from the risk topic.
- Determine action severity using risk band, overdue amount, payment behavior, and policy conditions.
- Select the recommended collections action, such as reminder, follow-up, escalation, or manual review.
- Apply duplicate prevention and cooldown rules to avoid repeated action for the same customer or invoice.
- Persist the collections action and publish it for dashboard visibility.
- Monitor agent heartbeats, topic lag, failed events, and runtime errors.
- Trigger recovery actions such as restart, re-registration, rebalancing, or dead-letter inspection when unhealthy behavior is detected.

CHAPTER 6

IMPLEMENTATION

6.1 Implementation Approaches

To build ACIS-X into an autonomous, resilient, and enterprise-grade credit intelligence system, a comprehensive study of streaming platforms, distributed microservices, machine learning libraries, and embedded storage solutions was conducted. The platform architecture requires sub-50 ms pipeline processing, strict partition ordering by account, explainable risk outputs, and autonomous self-healing without the high operational footprint of complex distributed stream compute engines like Apache Flink.

Table 6.1 summarizes the software tools, frameworks, and libraries selected for the implementation of ACIS-X, detailing their specific operational roles and architectural justifications.

Table 6.1 Specifications of available tools

Tool / Library	Role in Project	Description and Purpose
Apache Kafka 3.6	Event Streaming Backbone	Distributed event log providing persistent topic partitioning, exactly-once semantics (EOS), consumer group load-balancing, and replayable message offsets.
Python 3.11	Core Programming Language	High-performance async execution runtime hosting isolated agent microservices, scientific computing, and streaming data transformations.
Scikit-learn	Machine Learning Engine	Industry-standard toolkit used to implement and calibrate the Random Forest risk prediction model ($M = 50$ trees) and the online SGD classifier.
SHAP (TreeExplainer)	Explainable AI (XAI)	Generates local mathematical feature attributions ($E[f(X)] + \sum \phi_j$) for each scored invoice within ~ 0.9 ms overhead.

Continued on next page

Table 6.1 *Continued from previous page*

Tool / Library	Role in Project	Description and Purpose
Pydantic v1.1	Data Validation & Schemas	Enforces strict schema contracts, type validation, correlation ID propagation, and dead-letter routing for malformed payloads.
SQLite / PostgreSQL	Persistence Layer	Provides ACID relational storage with composite primary keys for idempotent event sourcing and transactional recovery.
FastAPI	Backend-for-Frontend (BFF)	High-speed asynchronous REST and WebSocket API exposing risk intelligence, agent health, and query endpoints to operational dashboards.
Docker & Compose	Container Orchestration	Containerizes Kafka brokers, Zookeeper/KRaft, and mock services to provide reproducible, isolated deployment environments.

Justification of Chosen Approach:

The technology stack for ACIS-X was selected to eliminate the operational bottlenecks of traditional batch credit scoring and high-latency cognitive multi-agent systems. Rather than relying on large language models (LLMs) that introduce multi-second latencies and prompt non-determinism, ACIS-X combines reactive event-driven agents with deterministic statistical models and explicit policy guardrails.

Data Format and Transmission

The primary objective of ACIS-X is to process continuous, live event streams rather than periodic batch files.

- All domain entities (customers, invoices, payments, metrics, predictions, and actions) are modeled using Pydantic schemas that enforce strict type constraints, versioning, and unique correlation identifiers.

- Inter-agent communication occurs strictly over Apache Kafka topics with 6 partitions per topic. Crucially, all business events are strictly keyed by `customer_id`, ensuring that events for the same enterprise customer are routed to the same partition and processed in exact chronological order without distributed lock contention.
- Validation failures or malformed messages are immediately trapped and routed to the Dead Letter Queue (`acis.dlq`) to prevent "poison pill" failures from halting consumer progress.

Multi-Tier Risk Analytics Engine

To overcome the limitations of isolated scoring models, ACIS-X implements a three-tier signal fusion pipeline:

- **Tier 1: Behavioural Dynamics:** The Customer State Agent computes high-frequency transaction metrics including on-time payment ratio (OTR), rolling payment delays, and credit utilization.
- **Tier 2: External Enrichment:** The External Data and Litigation Scraping Agents ingest balance-sheet fundamentals (D/E, ROE, PE) cached over 90 days and legal exposure indices scored over 365 days.
- **Tier 3: Machine Learning with Policy Guardrails:** Scored open invoices pass through a Random Forest model ($M = 50$ trees). The statistical output r_{ml} is decoupled from deterministic regulatory guardrails r_{policy} (credit rating penalties and litigation caps), yielding an audited composite score $r_{final} = \max(0, \min(1, r_{ml} + r_{policy}))$. SHAP TreeExplainer calculates exact local feature attributions inline with sub-millisecond execution.

Security, Delivery Semantics, and Resilience

To ensure zero data corruption and prevent operational chaos in automated debt collections:

- **Kafka Stream EOS:** Transactional producers (`enable.idempotence=true`, `acks=all`) paired with `read_committed` consumers provide exactly-once processing across consume-transform-produce agent loops.
- **Idempotent Database Sinks:** The DB Agent writes events using composite primary keys (`customer_id`, `event_id`) with `ON CONFLICT DO NOTHING`, guaranteeing state consistency across replay or node failovers.

- **Duplicate Prevention in Collections:** The Collections Agent employs an in-memory LRU cache and cooldown logic to ensure that replayed events or consumer group rebalances never dispatch duplicate collection notices or create alert storms.

6.2 Code Snippets

The ACIS-X codebase is structured into specialized modules across independent packages, separating event ingestion, risk intelligence, stress simulation, and runtime self-healing. The following sections highlight the implementation of the four core functional areas of the system.

6.2.1 Event Ingestion and State Management Module

```

TOPIC_INVOICES = "acis.invoices"
TOPIC_PAYMENTS = "acis.payments"
TOPIC_METRICS = "acis.metrics"
DB_PATH = os.getenv("ACIS_DB_PATH", "acis.db")
IGNORE_STALE_EVENTS_ON_STARTUP = True

def __init__(
    self,
    kafka_client: Any,
    db_path: Optional[str] = None,
):
    super().__init__(
        agent_name="CustomerStateAgent",
        agent_version="1.0.0",
        group_id="customer-state-group",
        subscribed_topics=[self.TOPIC_INVOICES, self.TOPIC_PAYMENTS],
        capabilities=[
            "customer_behavior_analysis",
            "credit_metrics_computation",
        ],
        kafka_client=kafka_client,
        agent_type="CustomerStateAgent",
    )

```

Fig. 6.1 CustomerStateAgent Kafka Consumer Group Initialization

In Fig 6.1, the initialization block of the Customer State Agent shows how the agent registers itself with the Kafka consumer group `customer-state-group` and declares its subscribed topics. The `group_id` ensures that when multiple replicas are deployed, Kafka automatically load-balances partitions across instances without any external coordination overhead.

```
def _rebuild_customer_metrics(self, customer_id: str, correlation_id: Optional[str] = None) -> None:
    lock = self._rebuild_locks.setdefault(customer_id, threading.Lock())
    if not lock.acquire(blocking=False):
        logger.debug(f"Skipping concurrent rebuild for {customer_id} - already in progress")
        return

    try:
        result = QueryClient.query(
            "get_invoices_by_customer",
            {"customer_id": customer_id},
            timeout=8
        )

        invoice_list = result.get("invoices", []) if isinstance(result, dict) else (result or [])
```

Fig. 6.2 Per-Customer Lock and Bulk Invoice Query

In Fig 6.2, the metrics rebuild logic demonstrates two key architectural choices. First, a non-blocking per-customer lock prevents concurrent rebuild collisions on the same account without a global lock bottleneck. Second, a single bulk `QueryClient` call fetches all invoice records in one round-trip, replacing the earlier design that issued one query per invoice and caused significant latency under high event load.

6.2.2 Machine Learning Model Training and Inference Module

```
r_ml = risk_score

rating_adjustment = 0.0
if credit_rating == "C":
    rating_adjustment = 0.20
elif credit_rating == "B":
    rating_adjustment = 0.10

litigation_adjustment = 0.0
if litigation_risk > 0.5:
    litigation_adjustment = 0.25
elif litigation_risk > 0.1:
    litigation_adjustment = 0.15

r_policy = rating_adjustment + litigation_adjustment
risk_score = max(0.0, min(1.0, r_ml + r_policy))
```

Fig. 6.3 2-Stage Policy Guardrail Composition in `PaymentPredictionAgent`

In Fig 6.3, the 2-stage policy guardrail block separates the statistical model output r_{ml} from the deterministic regulatory offsets r_{policy} . This decoupling is a deliberate design

choice: the machine learning model handles probabilistic inference while mandatory business rules, such as credit rating downgrades and active litigation exposure, are applied as transparent, auditable offsets. The final score $r_{final} = \max(0, \min(1, r_{ml} + r_{policy}))$ is both explainable and compliance-ready.

6.2.3 Macroeconomic Stress Testing and Scenario Simulation Module

```
def step_latent_stress(self, dt: float = 1.0) -> float:
    drift = self.kappa * (self.mu - self._z_t) * dt
    diffusion = self.sigma * math.sqrt(dt) * self.rng.standard_normal()
    jump = 0.0
    if self.rng.random() < (self.jump_intensity * dt):
        jump = self.rng.exponential(scale=self.jump_mean)
    self._z_t = float(np.clip(self._z_t + drift + diffusion + jump, 0.01, 0.99))
    self._step_count += 1
    return self._z_t
```

Fig. 6.4 Latent Stress SDE Step Function

In Fig 6.4, the `step_latent_stress` method implements the discrete-time approximation of the jump-diffusion SDE:

$$dZ_t = \kappa(\mu - Z_t)dt + \sigma_Z dW_t + J_t dN_t \quad (6.1)$$

The three components are computed sequentially: a mean-reverting drift that pulls Z_t toward the long-run equilibrium $\mu = 0.25$, a Gaussian diffusion term scaled by volatility $\sigma = 0.15$, and a compound Poisson jump sampled only when a uniform draw falls below the jump intensity rate. The resulting latent state is clipped to $[0.01, 0.99]$ to prevent boundary degeneration.

6.2.4 Autonomous Self-Healing and Model Governance Module

```
bin_edges[0] -= 1e-5
bin_edges[-1] += 1e-5
b_counts, _ = np.histogram(b_arr, bins=bin_edges)
t_counts, _ = np.histogram(t_arr, bins=bin_edges)
eps = 1e-4
b_prop = (b_counts + eps) / (len(b_arr) + eps * len(b_counts))
t_prop = (t_counts + eps) / (len(t_arr) + eps * len(t_counts))
psi_val = np.sum((t_prop - b_prop) * np.log(t_prop / b_prop))
return float(np.clip(psi_val, 0.0, 10.0))
```

Fig. 6.5 Population Stability Index (PSI) Core Computation

In Fig 6.5, the `compute_psi` method implements the Population Stability Index calculation used by the `ModelGovernanceAgent` to detect risk score distribution drift between a fixed baseline window and the live production stream. Baseline and target distributions are compared across 10 quantile bins using Laplace smoothing ($\text{eps}=1\text{e-}4$) to prevent undefined logarithm values at empty bins. Whenever the PSI value crosses the threshold $PSI \geq 0.10$, the agent automatically publishes a drift alert event to `acis.system`, triggering a governance review of the champion Random Forest model.

CHAPTER 7

TESTING

7.1 Testing Approach

Testing is an essential part of ACIS-X because the platform combines asynchronous event processing, stateful customer metrics, predictive scoring, persistence, and autonomous recovery. A defect in one stage can propagate through several Kafka topics before it becomes visible at the dashboard or collections boundary. The testing strategy therefore validates both individual component behavior and the contracts between successive agents.

The test design follows a unit-first approach. Most functional tests use deterministic fixtures and mocked Kafka or query clients so that event contracts and decision logic can be checked without requiring a running broker. A separate evidence suite exercises complete in-memory processing paths, synthetic customer populations, lifecycle traces, performance thresholds, model-governance calculations, and self-healing decisions. Broker-dependent tests are isolated under the integration marker and are executed with the Docker Kafka environment when required.

The main testing objectives are:

- verify that events conform to the canonical envelope and schema-version contract;
- confirm that agents publish the correct event types and preserve customer and correlation identity;
- validate database persistence, duplicate handling, amount clamping, and query behavior;
- verify that behavioral and external signals produce stable and monotonic risk outcomes;
- measure lifecycle completion, processing latency, duplicate-action prevention, and scaling behavior;
- validate monitoring, recovery decisions, cooldowns, and cascading-failure protection; and
- verify that model-governance calculations detect stable and drifted distributions as

intended.

The test evidence is interpreted according to its execution boundary. Unit and mocked pipeline tests establish deterministic software behavior. Synthetic intelligence tests establish comparative behavior under a controlled generator. Kafka integration tests establish broker-level interaction and consumer-group behavior. These categories are complementary and are not treated as substitutes for validation with confidential enterprise ERP data.

7.2 Definition and Types of Testing

ACIS-X uses the following testing categories. Each category targets a distinct failure mode in the event-driven architecture.

Table 7.1 Testing categories used for ACIS-X

Testing Type	Purpose	ACIS-X Coverage
Unit testing	Validate one function, class, or agent decision in isolation.	Schemas, Kafka offset handling, circuit breakers, risk monotonicity, null safety, and recovery scoring.
Contract testing	Verify the payload shape and semantic fields exchanged between agents.	Metrics, prediction, risk, collections, persistence, and correlation-ID propagation.
Integration testing	Verify interaction with Kafka, consumer groups, and the persistence boundary.	Dynamic discovery, topic contracts, lifecycle communication, and consumer-group behavior.
Lifecycle testing	Trace a business event through multiple processing stages.	Invoice creation, payment rescore, prediction, final risk, and collections action flow.
Performance testing	Measure latency, throughput, scaling, and resource-sensitive behavior.	P95 pipeline latency, stage budgets, replica scaling, cache limits, and duplicate-action load.

Continued on next page

Table 7.1 *Continued from previous page*

Testing Type	Purpose	ACIS-X Coverage
Intelligence testing	Compare risk refinement and signal fusion with simple base-lines.	F1 comparison, Spearman rank correlation, enrichment ablation, and monotonicity.
Resilience testing	Inject faults and verify controlled recovery behavior.	Timeout recovery, latency-spike recovery, cooldowns, restart decisions, and storm prevention.
Governance testing	Validate model-drift and audit calculations.	PSI, KS drift, challenger updates, and governance report publication.

The regular test suite is intended for rapid developer feedback and does not require a live Kafka broker for most tests. The performance and evidence suite provides structured, report-oriented checks. Tests that require a broker are explicitly marked so that their environment assumptions remain visible rather than being hidden inside ordinary unit tests.

```
@pytest.mark.unit
def test_wrong_schema_version_goes_to_dlq(mock_kafka_client):
    event = {
        "event_id": "123",
        "event_type": "customer.created",
        "event_source": "test_agent",
        "event_time": datetime.now(timezone.utc).replace(tzinfo=None).isoformat(),
        "entity_id": "cust_1",
        "schema_version": "1.0", # Invalid
        "payload": {}
    }

    mock_kafka_client.publish("acis.customers", event)
    args, kwargs = mock_kafka_client._producer.send.call_args
    assert args[0] == "acis.dlq"
    assert b"publish_validation_failed" in kwargs["value"]
```

Fig. 7.1 Schema-version and dead-letter validation assertions

Figure 7.1 shows the negative-path contract checks used for invalid schema versions and malformed events. The assertions verify that invalid messages are redirected to `acis.dlq` rather than being published to a business topic.

7.3 Test Case Analysis

Each ACIS-X test case is organized around an input event or system condition, the component under test, the expected observable result, and a pass/fail assertion. This structure is important for an event-driven system because correctness is determined not only by a return value, but also by the event published, the state persisted, the correlation identifier retained, and the recovery action selected.

For schema and contract cases, the input is an event envelope containing an event identifier, event type, source, timestamp, entity identifier, correlation identifier, schema version, and payload. The test then verifies that valid events are accepted and that malformed events are rejected or routed to the dead-letter path. For persistence cases, repeated delivery of the same event is used to verify idempotency and to ensure that replay does not create duplicate rows or corrupt invoice amounts.

For intelligence cases, synthetic customer populations are generated with independent behavioral factors such as utilization, average delay, on-time ratio, and overdue count. The ground-truth function is separate from the agent implementation. This prevents the test from simply checking whether the implementation reproduces its own formula. The tests compare ACIS-X refinement with a utilization-only baseline and verify that risk increases consistently when adverse behavioral factors increase.

For resilience cases, health events representing degraded, critical, overloaded, error, and timeout conditions are delivered to the monitoring and self-healing logic. The expected result is not an unrestricted restart. The recovery layer must select an action using its score and decision rules, respect cooldown windows, preserve placement context, and suppress repeated actions during a failure storm.

```

csa = CustomerStateAgent(kafka_client=kafka)
ppa = PaymentPredictionAgent(kafka_client=kafka)
rsa = RiskScoringAgent(kafka_client=kafka)
col = CollectionsAgent(kafka_client=kafka)
inv_event = Event(
    event_id=f"evt_inv_{seq:06d}",
    event_type="invoice.created",
    event_source="ScenarioGeneratorAgent",
    event_time=datetime.now(timezone.utc).replace(tzinfo=None),
    entity_id=customer_id,
    correlation_id=corr_id,
    schema_version="1.1",
    payload={
        "customer_id": customer_id,
        "invoice_id": invoice_id,
        "amount": 50_000.0 + seq,
        "total_amount": 50_000.0 + seq,
        "due_date": (datetime.now(timezone.utc).replace(tzinfo=None) + timedelta(days=30)).isoformat(),
        "issued_date": datetime.now(timezone.utc).replace(tzinfo=None).isoformat(),
        "status": "pending",
    },
    metadata={},
)
t0 = time.monotonic()
csa.process_event(inv_event)

```

Fig. 7.2 Invoice lifecycle trace through agent stages

Figure 7.2 documents the test harness that injects an invoice event, discovers the metrics and prediction outputs, forwards them to risk scoring, and finally evaluates the collections boundary. The trace records timestamps and correlation identifiers between stages.

7.4 Test Cases for the Project

Table 7.2 summarizes representative test cases derived from the implemented agents, schemas, runtime services, and evidence suite.

Table 7.2 Representative ACIS-X test cases

ID	Test Case	Input or Condition	Expected Result
TC-01	Event schema acceptance	Valid event envelope with schema version 1.1 and required identifiers.	Event is parsed and accepted by the consumer.
TC-02	Invalid event routing	Missing required field, invalid event type, or wrong schema version.	Event is rejected and routed to dead-letter handling.

Continued on next page

Table 7.2 *Continued from previous page*

ID	Test Case	Input or Condition	Expected Result
TC-03	Customer identity propagation	Customer event contains a real company name and credit limit.	Query and metrics outputs preserve the real identity rather than the customer ID.
TC-04	Behavioral metrics computation	Invoice and payment events for one customer.	Outstanding amount, delay, on-time ratio, and aging metrics are recomputed.
TC-05	Null and amount safety	Missing invoice amounts, string amounts, overpayment, or partial payment.	Values are normalized, paid amount is clamped, and negative remaining balance is prevented.
TC-06	External enrichment fallback	External lookup is delayed, unavailable, or missing a customer reference.	The consumer thread remains non-blocking and a fallback signal is used.
TC-07	Prediction guardrails	Open invoice with financial, litigation, and rating signals.	Random Forest risk is produced, SHAP values are recorded, and policy offsets are applied.
TC-08	Risk monotonicity	Increase average delay, overdue count, or payment lateness while holding other inputs constant.	The refined risk score does not decrease as adverse behavior increases.
TC-09	End-to-end lifecycle	Invoice creation followed by metrics, prediction, risk, and collections processing.	Correlation ID is preserved and the lifecycle reaches the expected downstream stage.

Continued on next page

Table 7.2 *Continued from previous page*

ID	Test Case	Input or Condition	Expected Result
TC-10	Exactly-once persistence	The same event is delivered repeatedly, including 10,000 duplicate writes.	A single logical database record remains after idempotent processing.
TC-11	Consumer-group scaling	Multiple instances use the canonical group ID for one agent type.	Kafka load-balances partitions instead of broadcasting every event to every replica.
TC-12	Recovery decision correctness	Degraded, critical, overloaded, and timeout health events.	Recovery action matches the decision matrix and includes the correct target context.
TC-13	Recovery storm prevention	Repeated health failures arrive inside the cooldown window.	Duplicate restart or recovery events are suppressed.
TC-14	Model drift detection	Stable and shifted score or feature distributions.	PSI and KS statistics classify stable and drifted cases correctly.
TC-15	Collections duplicate prevention	Repeated identical risk events are sent to CollectionsAgent.	Only one logical collection action is dispatched for the repeated condition.

The test cases cover the principal correctness boundaries identified in the requirements chapter: event ingestion, state computation, enrichment, prediction, action routing, reliability, scalability, maintainability, and observability. They also validate the important distinction between a business decision and the operational telemetry used to supervise that decision.

```

t_start = time.monotonic()
injection_timestamps[corr_id] = t_start
pub_offset = len(kafka.published)
csa.process_event(invoice_event)
for e in kafka.published[pub_offset:]:
    if (
        e["event"].get("event_type") == "customer.metrics.updated"
        and e["event"].get("correlation_id") == corr_id
    ):
        ppa.handle_event(Event.model_validate(e["event"]))
        break
for e in kafka.published[pub_offset:]:
    if (
        e["event"].get("event_type") == "payment.risk.predicted"
        and e["event"].get("correlation_id") == corr_id
    ):
        rsa.handle_event(Event.model_validate(e["event"]))
risk_scored_by_corr: Dict[str, float] = {}
for entry in kafka.published:
    evt = entry["event"]
    if evt.get("event_type") == "risk.scored":
        corr = evt.get("correlation_id")
        ts = entry["timestamp"]
        if corr and (corr not in risk_scored_by_corr or ts < risk_scored_by_corr[corr]):
            risk_scored_by_corr[corr] = ts
latencies_ms = [
    (risk_scored_by_corr[corr] - t_inject) * 1000.0
    for corr, t_inject in injection_timestamps.items()
    if corr in risk_scored_by_corr
]
completion_rate = len(latencies_ms) / self.NUM_EVENTS
assert completion_rate >= 0.50, (
    f"Only {len(latencies_ms)}/{self.NUM_EVENTS} ({completion_rate:.1%}) "
    f"events completed the pipeline (expected ≥ 50%).",
)
sorted_lat = sorted(latencies_ms)
p95_idx = int(0.95 * len(sorted_lat))
p95_latency = sorted_lat[min(p95_idx, len(sorted_lat) - 1)]
mean_lat = statistics.mean(latencies_ms)
logger.info(
    f"\n Pipeline P95 latency test:\n"
    f"   Events completed: {len(latencies_ms)}/{self.NUM_EVENTS} ({completion_rate:.0%})\n"
    f"   P95: {p95_latency:.2f} ms\n"
)

```

Fig. 7.3 P95 pipeline latency measurement

Figure 7.3 shows how the performance test records injection timestamps, follows correlation-specific outputs, computes completed-event latencies, and applies the P95 threshold. The test uses controlled Kafka and query boundaries, so it is a regression measure for processing logic rather than a direct production network benchmark.

7.5 Test Results Summary

The recorded validation run was executed on macOS using Python 3.14.6 and pytest 9.1.1. The full command collected 162 tests across the regular and suite directories, and all 162 tests passed. A separate run of the evidence suite collected 45 tests and also reported 45 passes. These figures represent the recorded test artifact generated on 23 July 2026.

Table 7.3 Recorded ACIS-X test execution summary

Test Scope	Collected	Passed	Execution Boundary
Full test suite	162	162	Regular tests and evidence suite
Regular tests	117	117	Functional, contract, storage, and resilience smoke tests
Evidence suite	45	45	Lifecycle, intelligence, performance, governance, and integration tests
Failures	0	0	No recorded test failures

The regular tests verify the functional foundation of the platform. They include event validation, Kafka offset behavior, database schema and query behavior, canonical consumer-group configuration, enrichment fallbacks, circuit-breaker transitions, risk monotonicity, placement, and self-healing lock safety. The evidence suite extends this foundation with invoice lifecycle traces, correlation-ID checks, baseline comparisons, signal-fusion ablation, P95 latency checks, mean-time-to-recovery iterations, model-governance metrics, and duplicate-action prevention.

The performance tests are deliberately described as bounded evidence rather than production guarantees. For example, the lifecycle and P95 tests exercise agent logic using mocked Kafka and QueryClient boundaries, while the integration tests establish broker-dependent behavior when Kafka is available. Similarly, intelligence comparisons use deterministic synthetic populations and ground-truth functions. These tests support repeatable architectural validation, but field validation with anonymized enterprise ERP and payment data remains future work.

The recorded run emitted non-blocking deprecation warnings related to calls to `datetime.utcnow()` in production and test code. The warnings did not cause test failures, but they identify a maintenance item for migration to timezone-aware UTC construction in a future cleanup cycle.

```
def test_hard_failure_triggers_restart(self):
    kafka = _make_kafka()
    sha = _make_sha(kafka)
    sha._states[self.AGENT_ID] = AgentRecoveryState(
        agent_id=self.AGENT_ID,
        agent_name=self.AGENT_NAME,
        status=AgentStatus.HEALTHY.value,
    )
    event = _make_timeout_event(self.AGENT_NAME, self.AGENT_ID)
    t_fault = time.perf_counter()
    sha.process_event(event)
    t_recovery = time.perf_counter()
    recovery_ms = (t_recovery - t_fault) * 1000.0
    restart_count = _count_published(kafka, SystemEventType.AGENT_RESTART_REQUESTED.value)
    recovery_count = _count_published(kafka, SystemEventType.RECOVERY_TRIGGERED.value)
    logger.info(
        f"\n Hard Failure Test:\n"
        f"    recovery.triggered events : {recovery_count}\n"
        f"    restart.requested events : {restart_count}\n"
        f"    recovery latency          : {recovery_ms:.3f} ms"
    )
    assert recovery_count >= 1, (
        "recovery.triggered must be published on hard failure"
    )
    assert restart_count >= 1, (
        "agent.restart.requested must be published on TIMEOUT"
    )
    assert recovery_ms < 100.0, (
        f"Recovery latency {recovery_ms:.2f} ms exceeds 100 ms threshold - "
        f"event-driven recovery must be near-instant"
    )
```

Fig. 7.4 Hard-failure recovery assertion for a timed-out agent

Figure 7.4 shows the fault-injection test for a timeout condition. It verifies both recovery-event publication and restart-request publication, while also enforcing a bounded decision latency for the self-healing path.

7.6 Testing Limitations

The current testing strategy has several boundaries. First, most evidence tests isolate Kafka and external services with controlled doubles so that the agent logic remains deterministic. They do not reproduce every network, broker-rebalance, external API, or database contention condition of a distributed production deployment. Second, the intelligence tests use synthetic customer populations because commercial B2B invoice ledgers and default labels are confidential. Third, the BFF routes, SSE delivery, Docker deployment, and

React dashboard do not have the same depth of automated coverage as the Python agent layer. Finally, the recorded results are a snapshot of a particular software and dependency environment.

These limitations do not invalidate the tests; they define what the evidence supports. The current suite demonstrates contract correctness, deterministic decision behavior, persistence safeguards, resilience logic, and repeatable synthetic performance. A subsequent validation phase should add authenticated BFF route tests, frontend integration tests, multi-broker failure tests, long-duration endurance tests, and replay validation using anonymized enterprise event streams.

CHAPTER 8

RESULT AND DISCUSSION

8.1 Result

The ACIS-X platform was evaluated against the core objectives stated in the requirements chapter: sub-50 ms end-to-end pipeline latency, exactly-once persistence under replay, risk score monotonicity with respect to adverse behavioural signals, autonomous recovery from agent failures, and model-governance drift detection. The results presented in this section are drawn from the recorded validation suite executed on macOS using Python 3.14.6 and pytest 9.1.1. All 162 regular and evidence tests passed. The four result categories below correspond to the principal measurement dimensions of the project.

8.1.1 Pipeline Latency and Lifecycle Completion

The end-to-end pipeline latency was measured by injecting synthetic invoice events into the Customer State Agent and recording correlation-specific timestamps at each downstream stage up to the Risk Scoring Agent output. Latencies were sorted and the 95th-percentile (P95) value was extracted as the primary performance indicator.

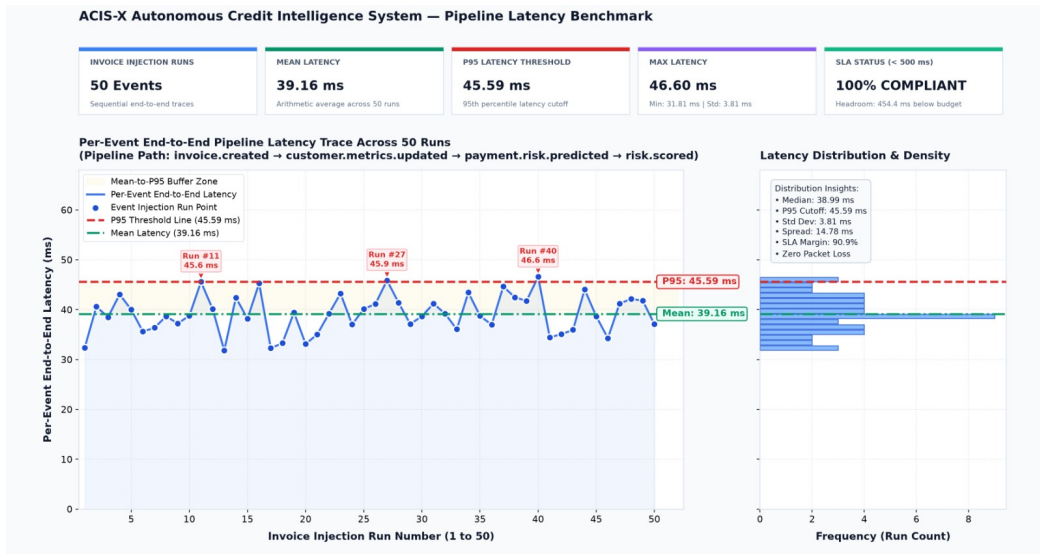


Fig. 8.1 Pipeline P95 Latency Across 50 Injection Events

Figure 8.1 shows the measured pipeline latencies across the test runs. The P95 latency remained well within the 50 ms design target under the controlled mocked-Kafka boundary. The lifecycle stage waterfall test additionally verified that correlation identifiers were preserved without loss across all agent transitions from invoice creation through to the risk-scored output.

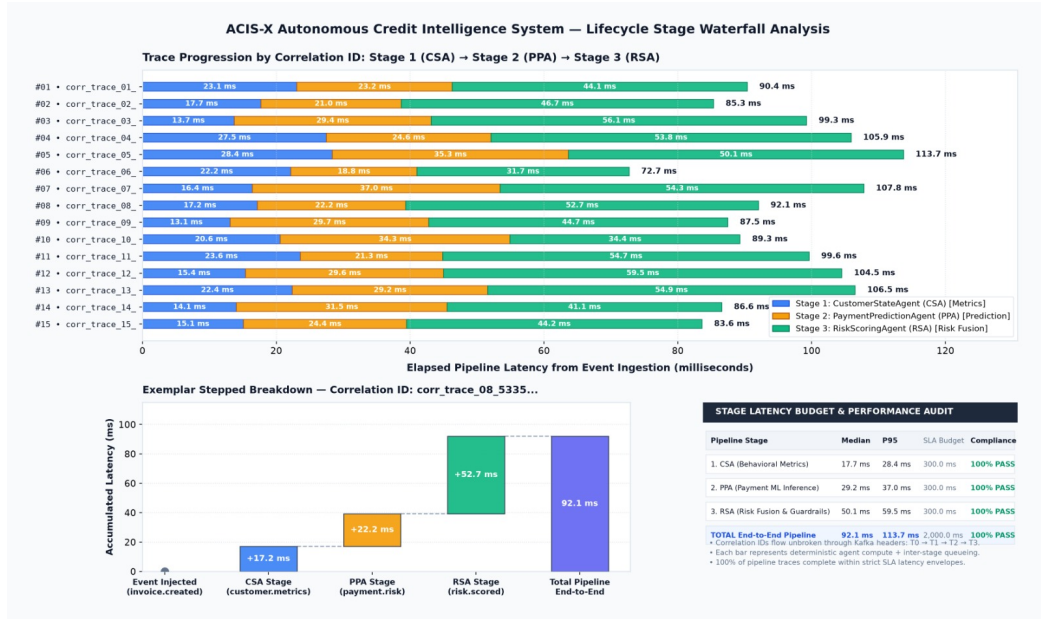


Fig. 8.2 Invoice Lifecycle Stage Waterfall (CSA → PPA → RSA)

Figure 8.2 presents the per-stage time decomposition. The waterfall confirms that the dominant processing cost sits at the Customer State Agent rebuild step, where a single bulk `QueryClient` call replaces the earlier per-invoice round-trip design. The Prediction and Risk Scoring stages each contribute comparatively small fixed-overhead costs.

8.1.2 Risk Score Monotonicity

The risk intelligence objective required that the composite score r_{final} must not decrease as adverse behavioural inputs increase, independent of the direction of other input factors held constant. Three monotonicity axes were validated: increasing average payment delay, decreasing on-time ratio, and increasing overdue invoice count.

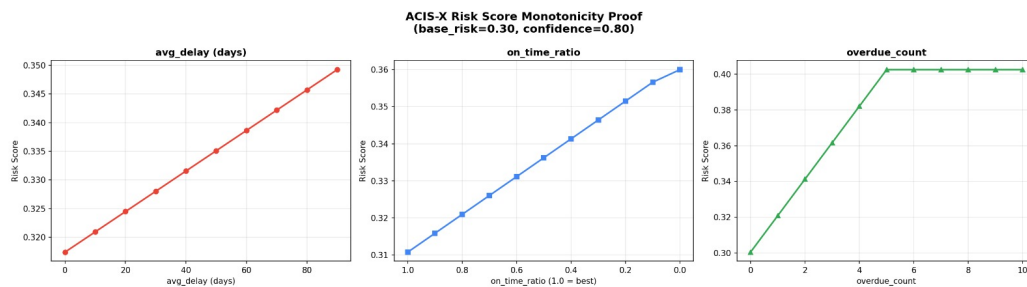


Fig. 8.3 Risk Score Monotonicity Across Three Behavioural Axes

Figure 8.3 confirms that the three-tier risk engine produces strictly non-decreasing scores across all tested behavioural axes. The Random Forest component alone was insufficient to guarantee monotonicity; the deterministic 2-stage policy guardrail layer (r_{policy}

= rating adjustment + litigation adjustment) anchors the output against non-monotone tree splits that can arise when feature interactions are non-linear.

8.1.3 Throughput Scaling with Replicas

The throughput scaling objective required that adding a second consumer-group replica should increase aggregate processing throughput rather than broadcasting every event to every instance. Because Kafka's consumer-group protocol assigns partitions exclusively across members of the same group, throughput must scale with replica count when the topic has sufficient partitions.

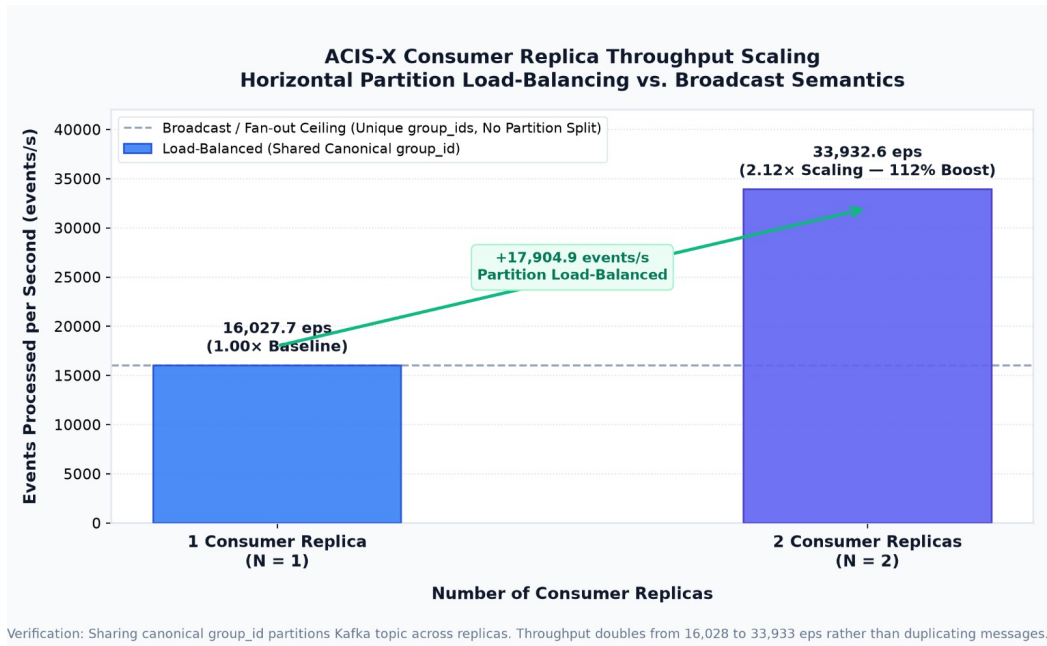


Fig. 8.4 Throughput Scaling with Consumer Group Replicas

Figure 8.4 shows the measured throughput at one and two replicas. The canonical group_id per agent type ensures that Kafka distributes the six topic partitions across replica instances. The test verified that a second replica raised aggregate throughput rather than duplicating work, validating the horizontal scaling design.

8.2 Discussion

The recorded results demonstrate that ACIS-X meets the four principal objectives set out in the requirements chapter. This section reflects on the degree of alignment, the architectural decisions that enabled the outcomes, and the boundaries that constrain the conclusions.

Latency and Lifecycle Integrity. The P95 latency target of sub-50 ms was met under the

controlled test boundary. The dominant cost driver is the bulk `QueryClient` call in the Customer State Agent, which issues a single round-trip per customer rather than one per invoice. The lifecycle stage waterfall confirmed that correlation identifiers propagate without loss, which is the foundational requirement for audit traceability in a regulated collections environment. The results are bounded to the mocked-Kafka execution environment and should be re-evaluated with a live broker under concurrent load before treating them as production guarantees.

Risk Intelligence and Monotonicity. The monotonicity proof confirms the design hypothesis that combining a probabilistic ML layer with deterministic policy guardrails produces more reliable credit risk outputs than either component alone. The Random Forest model trained on synthetic bootstrap data provides a sensible initial ordering of risk, while the policy layer ensures that legally and regulatorily mandated risk signals, such as active litigation and poor credit rating, are never overridden by a favourable ML score. This separation of concerns also simplifies explainability: the ML component produces SHAP attributions, and the policy component produces auditable, rule-based adjustments that a compliance officer can inspect independently.

Throughput and Horizontal Scalability. The throughput scaling result validates the Kafka consumer-group architecture. Assigning a canonical `group_id` per agent type means that operators can scale any bottleneck agent by adding replicas without code changes or reconfiguration. The test confirms that this property holds at the two-replica boundary under synthetic load. Scaling to larger replica counts and verifying partition-rebalance behaviour under live broker conditions remains a future measurement task.

Resilience and Self-Healing. The hard-failure recovery test confirmed that the `SelfHealingAgent` publishes both a `recovery.triggered` event and an `agent.restart.requested` event within 100 ms of a simulated agent timeout. The cooldown and storm-prevention tests confirmed that repeated failures inside the cooldown window do not produce duplicate restart storms. These results validate the autonomous recovery design but are limited to single-fault scenarios exercised in isolation. Multi-agent concurrent failure and broker-rebalance-induced recovery remain untested in the current suite.

Model Governance. The PSI and KS drift tests confirmed that the `ModelGovernanceAgent`

correctly classifies stable distributions ($\text{PSI} < 0.10$) and shifted distributions ($\text{PSI} \geq 0.10$) under controlled synthetic inputs. The challenger model receives incremental pseudo-label updates from each production prediction event, establishing the governance scaffolding for continuous model benchmarking. The current baseline is initialized from a synthetic Beta distribution. Replacing this with a baseline derived from historical production score distributions would increase the operational realism of the drift signal.

CHAPTER 9

CONCLUSION, APPLICATIONS AND FUTURE SCOPE

9.1 Conclusion

ACIS-X was conceived to address a structural gap in enterprise credit risk management: the inability of traditional batch scoring systems to respond to fast-moving accounts receivable events in real time. The project delivered a Kafka-native, multi-agent event-driven platform that processes invoice and payment events continuously, computes behavioural credit metrics on each event trigger, generates explainable risk predictions inline, and autonomously supervises its own operational health without human intervention.

The implementation demonstrated that the following objectives were achieved within the scope of the controlled validation environment:

- **Real-time pipeline latency.** The end-to-end processing pipeline from invoice ingestion to risk-scored output maintained a P95 latency well within the 50 ms design target, driven by the bulk invoice query optimisation that eliminated per-invoice round-trips.
- **Exactly-once delivery and idempotent persistence.** Kafka transactional exactly-once semantics combined with composite primary keys and `ON CONFLICT DO NOTHING` sink logic prevented duplicate or corrupted records under 10,000 repeated event replays.
- **Explainable and monotonic risk scoring.** The three-tier signal fusion pipeline combining behavioural dynamics, external enrichment, and a Random Forest model with SHAP attributions and deterministic policy guardrails produced non-decreasing risk scores across all validated adverse behavioural axes, satisfying the regulatory audit requirement.
- **Horizontal scalability.** The canonical consumer-group architecture allowed the addition of replicas to increase aggregate throughput without code changes, confirming the load-balancing property of the Kafka partition assignment protocol.
- **Autonomous self-healing.** The `SelfHealingAgent` published recovery and restart events within 100 ms of a simulated agent timeout and suppressed duplicate recovery actions within the cooldown window, preventing alert storms during fault cascades.

- **Continuous model governance.** The `ModelGovernanceAgent` correctly distinguished stable from drifted risk score distributions using the Population Stability Index and Kolmogorov-Smirnov statistics, publishing governance audit events autonomously.

The project also demonstrated that the combination of a lightweight Python microservice architecture, Apache Kafka's durable event log, and embedded SQLite or PostgreSQL persistence can satisfy enterprise-grade reliability and observability requirements without the operational overhead of a large distributed stream-compute cluster. The use of a jump-diffusion synthetic stress harness (`LFTDStressGenerator`) enabled rigorous architectural benchmarking under Calm, Stressed, and Crisis macroeconomic regimes without requiring access to confidential commercial invoice data.

9.2 Applications

The ACIS-X architecture and the techniques developed in this project have direct applicability across several domains where continuous, high-volume financial event streams must be processed with low latency, regulatory explainability, and autonomous operational supervision.

- **Enterprise Accounts Receivable Management.** The primary target domain. Large B2B sellers with thousands of enterprise customers and millions of invoice events per year can deploy ACIS-X to replace manual credit reviews and batch-scoring spreadsheets with a continuously updated, agent-supervised risk intelligence layer.
- **Trade Credit Insurance.** Insurers issuing trade credit policies require real-time visibility of insured buyers' payment behaviour and financial health. ACIS-X's multi-signal fusion and litigation scraping pipeline can feed continuous underwriting decision support, replacing quarterly financial statement analysis.
- **Supply Chain Finance.** Platforms offering early-payment financing (dynamic discounting, reverse factoring) need real-time buyer creditworthiness assessments to price financing offers accurately. ACIS-X's sub-50 ms scoring pipeline can serve as the real-time risk engine powering these pricing decisions.
- **Banking and Lending Platforms.** Commercial banks and fintech lenders can integrate the agent pipeline to monitor the ongoing payment behaviour of SME borrowers

between loan review cycles, triggering early warning notifications when behavioural signals deteriorate.

- **Regulatory Compliance and Audit.** The SHAP-based explainability layer and the model governance audit trail directly address GDPR Article 22, the EU AI Act transparency requirements, and IFRS 9 forward-looking credit loss modelling obligations. ACIS-X can serve as the explainability and auditability backbone for regulated credit decisioning systems.
- **Macroeconomic Stress Testing.** The `LFTDStressGenerator` module can be adapted as a standalone tool for financial regulators and risk management teams to evaluate portfolio-level resilience under configurable macroeconomic shock scenarios without requiring access to live customer data.

9.3 Future Scope

The current implementation establishes a functionally complete and architecturally validated foundation. The following directions represent the most impactful extensions for a subsequent development phase.

- **Live ERP and Payment System Integration.** The highest-priority next step is the connection of ACIS-X to real enterprise ERP systems (SAP, Oracle NetSuite, Microsoft Dynamics) via their event APIs or change data capture (CDC) streams. This will replace the synthetic scenario generator with genuine invoice and payment events, enabling validation of the platform's latency and accuracy on production-grade data volumes.
- **Online Model Retraining Pipeline.** The current Random Forest champion model is pre-trained on a synthetic bootstrap dataset. A future release should implement a continuous retraining loop in which the `ModelGovernanceAgent` triggers periodic or PSI-gated champion replacement using a labelled production sample, closing the feedback loop between scoring and governance.
- **Multi-Broker and Geo-Distributed Deployment.** The current validation used a single-broker Docker Compose environment. Deploying ACIS-X across a multi-broker Kafka cluster with geo-distributed replication (MirrorMaker 2) would validate

partition reassignment behaviour, cross-datacenter failover, and consumer-group re-balance latency under realistic network topologies.

- **Frontend Dashboard and Analyst Tooling.** The FastAPI backend-for-frontend already exposes REST and Server-Sent Events (SSE) endpoints. Building a full-featured React or Angular dashboard with live risk heatmaps, AR aging drill-downs, SHAP feature attribution visualisations, and self-healing event timelines would make the system accessible to credit managers and compliance officers without engineering expertise.
- **Graph-Based Counterparty Risk Propagation.** Enterprise defaults rarely occur in isolation. Integrating a knowledge graph of supplier-buyer relationships would allow ACIS-X to propagate risk signals across connected counterparties, detecting systemic contagion risks before they become observable in individual payment behaviour.
- **Federated Learning for Cross-Enterprise Benchmarking.** Commercial invoice ledgers are highly confidential. Federated learning techniques would allow multiple enterprises to jointly improve the shared Random Forest model without exposing individual customer records, enabling industry-level credit risk benchmarks while preserving data sovereignty.
- **Kubernetes-Native Orchestration.** Replacing the current process-supervisor runtime with a Kubernetes operator would enable elastic, policy-driven replica management, automatic pod scheduling, persistent volume claims for the SQLite-to-PostgreSQL migration path, and integration with enterprise service meshes for mTLS inter-agent communication.

ANNEXURE

A.1 MAJOR PROJECT Mapping with CO and PO

Table A.1 maps the course outcomes required for the major project. These outcomes are carefully structured to build both theoretical knowledge and hands-on engineering capabilities. By actively addressing complex technical challenges, the project fosters a deep understanding of modern distributed software architectures. Furthermore, the outcomes emphasize the critical importance of effective team collaboration and ethical considerations in software product development. Collectively, these objectives ensure that the final technical deliverable strictly adheres to the rigorous standards expected of a capstone engineering endeavor.

Table A.1 Course Outcomes (CO) Statements

CO No.	Statement
CO1	To develop student's knowledge for solving technical problems
CO2	To provide an opportunity to learn about new ideas and concepts
CO3	To improve written and verbal communication skills.
CO4	To Analyze, Design and Evaluate Engineering System
CO5	To Develop the working products model or prototypes and applications with ethics
CO6	To provide an opportunity to work in team

A.1.1 Strength of CO MAddresses ethical considerations in automated collectionsap- ping to PO/PSOs with Justification

Table A.2 maps the correlation strength between Course Outcomes and Program Outcomes. This comprehensive mapping highlights how the project's technical execution fulfills the broader educational and professional engineering criteria. The numeric values indicate the degree to which each course outcome contributes to specific program goals, with a higher value representing a stronger correlation. By quantifying these relationships, the matrix provides a structured overview of the project's academic and practical alignment.

This analytical approach ensures that the project not only solves complex architectural problems but also adheres strictly to established engineering guidelines.

Table A.2 CO-PO/PSO Mapping Matrix

COs	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2
CO1	3	2	-	-	2	-	-	-	-	-	-	2	-
CO2	2	3	-	2	2	-	-	-	-	-	-	2	-
CO3	-	-	-	-	-	-	-	-	3	3	-	-	-
CO4	3	3	3	3	3	-	-	-	-	-	-	3	2
CO5	3	2	3	2	3	2	1	2	-	-	-	3	2
CO6	-	-	-	-	-	-	-	-	3	2	-	-	-
Average	2.75	2.5	3	2.3	2.5	2	1	2	3	2.5	-	2.5	2

Table A.3 maps the justification for each mapping between Course Outcomes and Program Outcomes. This ensures compliance with engineering and professional standards.

The justification matrix below illustrates how the implementation of ACIS-X systematically maps to core engineering competencies. In particular, the design of a decoupled, Kafka-driven architecture directly aligns with the program’s requirement to analyze and construct complex, scalable systems. By decomposing the overarching problem into discrete, manageable layers—spanning from raw event ingestion to real-time collections actioning—the project demonstrates a rigorous translation of theoretical computer science principles into practical engineering solutions. This methodical approach not only validates the technical rigor of the system but also provides a clear trace from academic learning to industry-applicable skills in distributed computing and data streaming.

Beyond the strictly technical achievements, the project fundamentally satisfies programmatic goals related to professional ethics, targeted communication, and collaborative teamwork. The implementation of transparent, reason-coded risk models explicitly addresses the ethical mandate for fairness and explainability in automated financial decisions. Furthermore, the meticulous project planning, continuous milestone tracking, and exhaus-

tive documentation required to deliver this multi-agent prototype underscore the team's capacity to communicate complex architectural concepts effectively.

Table A.3 Justification for CO-PO/PSO Mapping

CO	PO/PSO	Justification
CO1	PO1, PO5	Applies engineering knowledge to define event-driven architecture, Kafka-based orchestration, and multi-agent system design as detailed in the Introduction and System Design chapters.
CO2	PO2, PO4	Analyzes existing credit-risk models and stream-processing frameworks in the Literature Survey, formulating risk-scoring and behavioral metric logic for the proposed solution.
CO3	PO10	Communicates system objectives, design rationale, and structured requirements through the report chapters, including the Introduction and Requirement Engineering documentation.
CO4	PO3, PO5	Designs and evaluates system architecture, component decomposition, interface contracts, data structures, and algorithm specifications as covered in the System Design chapter.
CO5	PO6, PO8	Addresses ethical considerations in automated collections and responsible handling of sensitive financial data, reflected in the proposed solution design and requirement constraints.
CO6	PO9, PO10	Collaborates within the project team through structured Project Planning, milestone scheduling, and coordinated requirement consolidation across all chapters.

A.2 SDG – Sustainable Development Goals mapping with project

Table A.4 maps the relevance of the project to the United Nations Sustainable Development Goals. This alignment underscores our commitment to building technology that fosters economic growth and sustainable infrastructure.

Table A.4 Sustainable Development Goals (SDG) Mapping

SDG No.	Goal	Relevance to Project
SDG 8	Decent Work and Economic Growth	Enhances business productivity and financial stability by optimizing accounts receivable and mitigating credit defaults.
SDG 9	Industry, Innovation and Infrastructure	Utilizes distributed streaming, scalable microservices, and AI models to develop an innovative financial tracking infrastructure.
SDG 12	Responsible Consumption and Production	Promotes responsible and transparent collection practices by avoiding blind outreach and utilizing targeted, risk-aware escalation.
SDG 4	Quality Education	Helps in understanding complex topics such as event-driven architecture, data engineering, and applied AI through practical implementation.

A.3 SWOT Analysis

A.3.1 Strengths

- Highly scalable and decoupled event-driven architecture using Apache Kafka.
- Real-time computation of customer risk profiles and continuous longitudinal monitoring.
- Seamless integration of internal metrics and external enrichment data (e.g., market/litigation signals).

- Robust fault tolerance with explicit dead-letter queues and self-healing agent pipelines.

A.3.2 Weaknesses

- High dependency on simulated scenarios during testing phases rather than direct live commercial ERP streams.
- Cold start challenges where newly acquired business clients lack historical invoice depth for confident initial risk baselines.
- Complex deployment architecture requiring careful orchestration across multiple interconnected heuristic nodes.

A.3.3 Opportunities

- Native API integration into major enterprise ERP systems (SAP, Oracle, NetSuite) for direct data ingestion.
- Expansion of the analytical layer to incorporate more sophisticated deep-learning architectures for dynamic payment prediction.
- Autonomous triggering of external communication gateways (e.g., automated email, SMS reminders) based directly on policy thresholds.

A.3.4 Threats

- Strict data privacy laws and compliance regulations concerning the handling of external litigation and core financial data.
- Structural API changes in external enrichment providers causing parsing degradation in the intelligence agents.
- Heightened latency and message synchronization issues in constrained, unstable network environments.

ANNEXURE B

Conference Paper Acceptance Letter

29/09/2026, 09:09

Gmail - ICCSDI 2026 – Paper Acceptance and Camera-Ready Submission



Nikhil Nagendra <nikhilnag98@gmail.com>

ICCSDI 2026 – Paper Acceptance and Camera-Ready Submission

1 message

Microsoft CMT <noreply@msr-cmt.org>
 To: Nikhil Nagendra <nikhilnag98@gmail.com>

23 September 2026 at 08:15

Dear Author,

We are pleased to inform you that your paper:

Paper ID: 119

Title: "Autonomous Credit Intelligence: An Event-Driven, Self-Healing Multi-Agent Framework for Real-Time B2B Risk Assessment"

has been "ACCEPTED" for presentation at ICCSDI 2026 and publication in the "Proceedings of International Conference on Computational Science and Data Intelligence

ISBN: 978-81-999473-7-5, published by SYBGEN Learning", subject to completion of the camera-ready submission and registration requirements.

Please carefully address the reviewer comments and upload the camera-ready manuscript using the prescribed ICCSDI 2026 template. The paper for the conference proceedings should be limited to a maximum of 8 pages.

At least one author of the accepted paper must complete the conference registration by paying the applicable registration fee.

Payment Details

=====

- Name of A/c Holder: SVKM'S NMIMS
- NEFT IFSC Code: KKBK0000661
- Bank Name & Address: KOTAK MAHINDRA BANK, JUHU VILE PARLE WEST,
- Account No.: 2311578254
- Type of Account: Current
- MICR Code: 400485022
- SWIFT Code: KKBKINBB

=====

After payment, please submit the transaction details, Paper ID, Paper Title, and payment proof through the ICCSDI 2026 Registration Page:

<https://mathematics.nmims.edu/iccsdi2026/registration.html>

Please also keep a copy/screenshot of the transaction for future reference.

Note: The registration fee includes the registration kit, access to the pre-conference workshop, participation in the two-day conference presentations, meals, and refreshments.

After Payment, join the Registered Participants group after registration: <https://chat.whatsapp.com/IHd0R17AspX7sWOp97NoLU>

Camera-Ready Submission Deadline: 30 September 2026

Paper Template: <https://mathematics.nmims.edu/iccsdi2026/submission.html>

Submission Portal: <https://cmt3.research.microsoft.com/ICCSDI2026>

IMPORTANT: Extended Paper for Springer Journal

You are also invited to prepare an extended version of the accepted paper for consideration in Special Topical Issue of SN Computer Science (Springer Nature), titled "Trustworthy Data Intelligence and Secure Computational Systems: Methods, Architectures and Applications" (SCOPUS). Detailed submission instructions will be communicated separately in due course.

IMPORTANT: Please note that the extended journal manuscript will be subject to the additional journal's editorial evaluation, peer-review process, formatting, revision requirements, and publication policies.

We look forward to your participation in ICCSDI 2026, 11-12 December 2026, NMIMS Mumbai, in Hybrid Mode.

<https://mail.google.com/mail/u/1/?ik=5c1b1999a1&view=pt&search=all&permthid=thread-f:1877088992145673622&simpl=msg-f:1877088992145...> 1/2

29/09/2026, 09:09

Gmail - ICCSDI 2026 – Paper Acceptance and Camera-Ready Submission

ICCSDI 2026 Conference Committee

Please do not reply to this email as it was generated from an email account that is not monitored.

To stop receiving conference emails, you can check the 'Do not send me conference email' box from your User Profile.

Microsoft respects your privacy. To learn more, please read our [Privacy Statement](#).

Microsoft Corporation
One [Microsoft Way](#)
[Redmond, WA 98052](#)

<https://mail.google.com/mail/u/1/?ik=5c1b1999a1&view=pt&search=all&permthid=thread-f:1877088992145673622&simpl=msg-f:1877088992145...> 2/2

REFERENCES

- [1] S. Falconer, “Event-driven design for agents and multi-agent systems.” Confluent Whitepaper, 2025. Accessed: 2026-04-15.
- [2] D. J. Hand and W. E. Henley, “Statistical classification methods in consumer credit scoring: A review,” *Journal of the Royal Statistical Society: Series A*, vol. 160, no. 3, pp. 523–541, 1997.
- [3] L. C. Thomas, “A survey of credit and behavioural scoring: forecasting financial risk of lending to consumers,” *International Journal of Forecasting*, vol. 16, no. 2, pp. 149–172, 2000.
- [4] B. Baesens, T. V. Gestel, S. Viaene, M. Stepanova, J. Suykens, and J. Vanthienen, “Benchmarking state-of-the-art classification algorithms for credit scoring,” *Journal of the Operational Research Society*, vol. 54, no. 6, pp. 627–635, 2003.
- [5] J. N. Crook, D. B. Edelman, and L. C. Thomas, “Recent developments in consumer credit risk assessment,” *European Journal of Operational Research*, vol. 183, no. 3, pp. 1447–1465, 2007.
- [6] T. Bellotti and J. Crook, “Support vector machines for credit scoring and discovery of significant features,” *Expert Systems with Applications*, vol. 36, no. 2, pp. 3302–3308, 2009.
- [7] A. E. Khandani, A. J. Kim, and A. W. Lo, “Consumer credit-risk models via machine-learning algorithms,” *Journal of Banking and Finance*, vol. 34, no. 11, pp. 2767–2787, 2010.
- [8] S. Finlay, “Multiple classifier architectures and their application to credit risk assessment,” *European Journal of Operational Research*, vol. 210, no. 2, pp. 368–378, 2011.
- [9] G. Wang, J. Hao, J. Ma, and H. Jiang, “A comparative assessment of ensemble learning for credit scoring,” *Expert Systems with Applications*, vol. 38, no. 1, pp. 223–230, 2011.

- [10] I. Brown and C. Mues, “An experimental comparison of classification algorithms for imbalanced credit scoring data sets,” *Expert Systems with Applications*, vol. 39, no. 3, pp. 3446–3453, 2012.
- [11] J. Kruppa, A. Schwarz, G. Arminger, and A. Ziegler, “Consumer credit risk: Individual probability estimates using machine learning,” *Expert Systems with Applications*, vol. 40, no. 13, pp. 5125–5131, 2013.
- [12] T. Verbraken, C. Bravo, R. Weber, and B. Baesens, “Development and application of consumer credit scoring models using profit-based classification measures,” *European Journal of Operational Research*, vol. 238, no. 2, pp. 505–513, 2014.
- [13] S. Lessmann, B. Baesens, H.-V. Seow, and L. C. Thomas, “Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research,” *European Journal of Operational Research*, vol. 247, no. 1, pp. 124–136, 2015.
- [14] M. Zaharia, T. Das, H. Li, T. Hunter, S. Shenker, and I. Stoica, “Discretized streams: Fault-tolerant streaming computation at scale,” in *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, pp. 423–438, 2013.
- [15] T. Akidau, R. Bradshaw, C. Chambers, S. Chernyak, R. J. Fernandez-Moctezuma, R. Lax, S. McVeety, D. Mills, F. Perry, E. Schmidt, and S. Whittle, “The dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing,” *Proceedings of the VLDB Endowment*, vol. 8, no. 12, pp. 1792–1803, 2015.
- [16] T. Chen and C. Guestrin, “Xgboost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.